

MAKALAH
REINFORCEMENT LEARNING

Disusun untuk Memenuhi Tugas Mata Kuliah Pembelajaran Mesin

Dosen Pengampu:

Nurfalinda, S.T., M.Cs.



DISUSUN OLEH:

Ferdy Mulyadi Reo Goran	2301020016
Zoni Fatma Mulia	2301020020
Hakim Indra Lesmana	2301020024

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK DAN TEKNOLOGI KEMARITIMAN
UNIVERSITAS MARITIM RAJA ALI HAJI

2026

DAFTAR ISI

DAFTAR ISI.....	i
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah.....	3
1.3 Tujuan	3
BAB II TEORI DASAR REINFORCEMENT LEARNING.....	4
2.1 Pengertian Reinforcement Learning	4
2.2 Komponen Utama Reinforcement Learning	6
2.2.1 Agent (Agen).....	6
2.2.2 Environment (Lingkungan).....	6
2.2.3 State (Keadaan).....	7
2.2.4 Action (Tindakan)	8
2.2.5 Reward (Imbalan)	8
2.2.6 Siklus Interaksi Agent-Environment.....	10
2.3 Markov Decision Process (MDP)	11
2.3.1 Definisi Formal MDP.....	11
2.3.2 Markov Property (Sifat Markov)	12
2.4 Policy dan Value Function	13
2.4.1 Policy (Kebijakan) – π	13
2.4.2 State-Value Function – $V^{\pi(s)}$	14
2.4.3 Action-Value Function (Q-Function) – $Q^{\pi(s), a}$	14
2.4.4 Persamaan Bellman.....	15
2.5 Eksplorasi vs. Eksploitasi	16
2.6 Posisi Reinforcement Learning dalam Machine Learning.....	19
BAB III LANGKAH-LANGKAH DAN ALGORITMA REINFORCEMENT LEARNING	20
3.1 Kerangka Umum Algoritma Reinforcement Learning.....	20
3.2 Algoritma Q-Learning.....	22
3.2.1 Prinsip Kerja	22
3.2.2 Langkah-Langkah Algoritma Q-Learning	24
3.2.3 Contoh Ilustrasi Q-Learning	25
3.3 Algoritma SARSA	25

3.3.1 Prinsip Kerja	25
3.3.2 Langkah-Langkah Algoritma SARSA	26
3.3.3 Perbandingan Perilaku SARSA dan Q-Learning	27
3.4 Deep Q-Network (DQN).....	28
3.4.1 Motivasi dan Prinsip Kerja	28
3.4.2 Inovasi Kunci DQN	29
3.4.3 Langkah-Langkah Algoritma DQN	30
3.5 Perbandingan Algoritma	32
BAB IV STUDI KASUS	33
4.1 Studi Kasus 1: ChatGPT dan Reinforcement Learning from Human Feedback	33
4.1.1 Latar Belakang	33
4.1.2 Formulasi Masalah sebagai RL.....	33
4.1.3 Tahapan Pelatihan RLHF	34
4.1.4 Hasil	35
4.1.5 Keterkaitan dengan Teori	36
4.2 Studi Kasus 2: Reinforcement Learning pada Sistem Rekomendasi (YouTube).....	36
4.2.1 Latar Belakang	36
4.2.2 Formulasi Masalah sebagai RL.....	37
4.2.3 Mengapa RL Lebih Baik dari Supervised Learning untuk Rekomendasi?.....	38
4.2.4 Arsitektur dan Metode	38
4.2.5 Hasil	39
4.2.6 Keterkaitan dengan Teori	40
4.3 Keterkaitan Studi Kasus dengan Teori.....	41
BAB V PENUTUP.....	42
5.1 Kesimpulan	42
5.2 Saran.....	43
DAFTAR PUSTAKA	44

BAB I

PENDAHULUAN

1.1 Latar Belakang

Kecerdasan buatan (*Artificial Intelligence/AI*) telah mengalami perkembangan yang sangat pesat dalam beberapa dekade terakhir. Kemajuan ini didorong oleh peningkatan kapasitas komputasi, ketersediaan data dalam jumlah besar, serta terobosan-terobosan algoritmik yang memungkinkan komputer untuk melakukan tugas-tugas yang sebelumnya hanya dapat dilakukan oleh manusia. Salah satu cabang dari AI yang telah menarik perhatian besar dari para peneliti dan praktisi adalah *machine learning* (pembelajaran mesin), yaitu kemampuan komputer untuk belajar dari data dan pengalaman tanpa diprogram secara eksplisit untuk setiap skenario yang mungkin terjadi [1].

Di dalam *machine learning* sendiri, terdapat tiga paradigma utama yang masing-masing memiliki karakteristik dan pendekatan yang berbeda: *supervised learning* (pembelajaran terawasi), *unsupervised learning* (pembelajaran tak terawasi), dan *reinforcement learning* (pembelajaran penguatan) [1]. Dari ketiga paradigma tersebut, *reinforcement learning* (RL) menawarkan pendekatan yang paling mirip dengan cara manusia dan hewan belajar secara alami yaitu melalui interaksi langsung dengan lingkungan dan umpan balik dari konsekuensi tindakan yang diambil.

Reinforcement learning (RL) merupakan paradigma pembelajaran mesin yang berfokus pada bagaimana sebuah *agent* (agen) harus mengambil tindakan (*action*) dalam suatu lingkungan (*environment*) untuk memaksimalkan akumulasi *reward* (imbalan) dari waktu ke waktu [1]. Berbeda dengan *supervised learning* yang memerlukan dataset berlabel berisi pasangan input-output yang benar, RL tidak memiliki "guru" yang memberitahu jawaban benar. Sebaliknya, *agent* RL harus menemukan sendiri strategi terbaik melalui proses coba-coba (*trial-and-error*) dengan berinteraksi langsung terhadap lingkungannya. Pendekatan ini terinspirasi secara mendalam dari cara manusia dan hewan belajar melalui pengalaman: ketika seorang anak menyentuh api dan merasakan sakit, ia akan belajar untuk tidak mengulangnya; sebaliknya, ketika ia menerima pujian setelah berbuat baik, perilaku tersebut akan cenderung diulangi. Prinsip sederhana inilah bahwa tindakan yang menghasilkan konsekuensi positif

cenderung diperkuat dan tindakan yang menghasilkan konsekuensi negatif cenderung dihindari yang menjadi inti dari *reinforcement learning*.

Secara historis, akar dari RL dapat ditelusuri hingga beberapa dekade lalu. Konsep pengambilan keputusan optimal dalam kondisi ketidakpastian pertama kali diformalisasikan oleh Richard Bellman pada tahun 1950-an melalui pemrograman dinamis (*dynamic programming*) [6]. Sementara itu, dari sisi psikologi, teori pengkondisian operan (*operant conditioning*) yang dikembangkan oleh B.F. Skinner pada tahun 1930-an [4] memberikan landasan konseptual bahwa perilaku dapat dibentuk melalui penguatan (*reinforcement*) dan hukuman (*punishment*). Integrasi kedua tradisi inilah pemrograman dinamis dan psikologi perilaku yang melahirkan bidang *reinforcement learning* modern seperti yang kita kenal saat ini, sebagaimana dirangkum secara komprehensif dalam buku teks seminal Sutton dan Barto [1].

Keberhasilan RL dalam berbagai domain aplikasi telah menunjukkan potensinya yang luar biasa. Salah satu contoh paling menonjol dalam beberapa tahun terakhir adalah penggunaan teknik *Reinforcement Learning from Human Feedback* (RLHF) dalam melatih model bahasa besar seperti ChatGPT. Pada tahun 2022, OpenAI mempublikasikan makalah InstructGPT yang mendemonstrasikan bahwa RL dapat digunakan untuk menyelaraskan perilaku model bahasa agar menghasilkan respons yang lebih bermanfaat, jujur, dan aman sesuai preferensi manusia [2]. Selain itu, RL juga telah diterapkan dalam skala produksi pada sistem rekomendasi YouTube yang melayani lebih dari dua miliar pengguna, mengoptimalkan kepuasan pengguna jangka panjang alih-alih metrik sesaat seperti jumlah klik [3]. Pencapaian-pencapaian ini mendemonstrasikan bahwa RL tidak hanya relevan untuk domain permainan, tetapi juga mampu menyelesaikan permasalahan dunia nyata yang sangat kompleks dan berdampak luas.

Selain dua contoh di atas, RL juga telah diterapkan secara sukses dalam berbagai bidang lain seperti robotika (pelatihan robot untuk berjalan dan memanipulasi objek), kendaraan otonom (*self-driving cars*), permainan (*game AI* seperti DQN pada Atari dan AlphaGo pada permainan Go), pengelolaan sumber daya di pusat data, hingga penemuan obat baru [7]. Luasnya cakupan aplikasi ini menunjukkan bahwa RL merupakan kerangka kerja yang sangat general untuk memecahkan berbagai permasalahan pengambilan keputusan sekuensial.

Oleh karena itu, pemahaman yang mendalam mengenai teori dasar, prosedur algoritmik, dan aplikasi nyata dari *reinforcement learning* menjadi sangat penting bagi mahasiswa dan

praktisi di bidang kecerdasan buatan dan ilmu komputer. Makalah ini disusun untuk memberikan pemahaman komprehensif mengenai aspek-aspek tersebut, dimulai dari fondasi teori yang mendasari RL, dilanjutkan dengan penjelasan langkah demi langkah dari algoritma-algoritma kunci, dan diakhiri dengan studi kasus nyata yang mengilustrasikan bagaimana teori dan algoritma tersebut diterapkan untuk menyelesaikan masalah dunia nyata.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah yang akan dibahas dalam makalah ini adalah sebagai berikut:

1. Apa teori dasar yang mendasari *reinforcement learning*, termasuk konsep-konsep fundamental seperti *agent*, *environment*, *state*, *action*, *reward*, *Markov Decision Process*, *policy*, dan *value function*?
2. Bagaimana langkah-langkah dan prosedur algoritma dalam *reinforcement learning*, khususnya algoritma Q-Learning, SARSA, dan *Deep Q-Network* (DQN)?
3. Bagaimana penerapan *reinforcement learning* dalam studi kasus nyata yang relevan, dan bagaimana keterkaitan antara teori, algoritma, dan hasil penerapan tersebut?

1.3 Tujuan

Tujuan dari penulisan makalah ini adalah:

1. Menjelaskan secara mendalam teori dasar *reinforcement learning* meliputi konsep *agent*, *environment*, *state*, *action*, *reward*, *policy*, *value function*, *Markov Decision Process* (MDP), Persamaan Bellman, serta dilema eksplorasi vs. eksploitasi, sehingga pembaca memiliki fondasi pemahaman yang kuat untuk mempelajari algoritma-algoritma RL.
2. Menjabarkan secara rinci langkah-langkah dan prosedur algoritma *reinforcement learning*, khususnya Q-Learning, SARSA, dan *Deep Q-Network* (DQN), termasuk persamaan pembaruan, pseudocode, serta perbandingan karakteristik masing-masing algoritma.
3. Menyajikan studi kasus penerapan *reinforcement learning* yang relevan dan selaras dengan teori serta langkah-langkah yang telah dijabarkan, disertai analisis mendalam mengenai formulasi masalah, arsitektur sistem, hasil yang dicapai, dan keterkaitan langsung dengan konsep-konsep teori yang telah dibahas.

BAB II

TEORI DASAR REINFORCEMENT LEARNING

2.1 Pengertian Reinforcement Learning

Reinforcement learning (RL) adalah sebuah area dalam *machine learning* yang mempelajari bagaimana seorang *agent* (agen) yang cerdas harus mengambil serangkaian keputusan dalam suatu lingkungan (*environment*) untuk memaksimalkan gagasan kumulatif dari *reward* (imbalan) [1]. Menurut Sutton dan Barto [1], RL merupakan pendekatan komputasional untuk memahami dan mengotomatisasi pembelajaran berdasarkan tujuan (*goal-directed learning*) dan pengambilan keputusan. Dengan kata lain, RL adalah tentang bagaimana belajar *apa yang harus dilakukan* bagaimana memetakan situasi ke tindakan sedemikian rupa sehingga sinyal *reward* numerik yang diterima dimaksimalkan.

Untuk memahami RL secara intuitif, bayangkan seorang pemain catur pemula yang baru belajar bermain. Ia tidak memiliki buku panduan yang menjelaskan langkah terbaik untuk setiap situasi (seperti pada *supervised learning*). Sebaliknya, ia bermain berkali-kali, bereksperimen dengan berbagai langkah, dan belajar dari hasil akhir permainan menang atau kalah. Seiring berjalannya waktu, ia mulai mengenali pola: langkah-langkah tertentu di situasi tertentu cenderung mengarah pada kemenangan, sementara langkah lain sering berujung pada kekalahan. Tanpa diberitahu secara eksplisit "langkah terbaik pada posisi X adalah Y", ia secara bertahap mengembangkan intuisi dan strategi yang semakin baik. Proses inilah yang dimodelkan oleh *reinforcement learning*.

Ide dasar dari RL berasal dari psikologi perilaku, khususnya konsep pengkondisian operan (*operant conditioning*) yang dikembangkan oleh B.F. Skinner pada tahun 1938 [4]. Dalam eksperimen klasik Skinner, seekor tikus ditempatkan dalam sebuah kotak (*Skinner box*) dengan sebuah tuas. Ketika tikus secara tidak sengaja menekan tuas, ia menerima makanan (penguatan positif/*positive reinforcement*). Setelah beberapa kali pengalaman ini, tikus belajar bahwa menekan tuas menghasilkan makanan dan mulai menekan tuas dengan sengaja dan lebih sering. Sebaliknya, jika menekan tuas menghasilkan sengatan listrik ringan, tikus belajar untuk menghindari tuas tersebut. Prinsip dasar ini bahwa perilaku dibentuk oleh konsekuensinya merupakan inti dari *reinforcement learning* dalam konteks komputasional.

Penting untuk dipahami bagaimana RL berbeda secara fundamental dari dua paradigma *machine learning* lainnya, karena pemahaman ini menentukan kapan RL menjadi pendekatan yang tepat:

- **Supervised Learning (Pembelajaran Terawasi):** Dalam *supervised learning*, model belajar dari dataset yang berisi pasangan input-output yang telah berlabel. Misalnya, untuk tugas klasifikasi gambar, model diberikan ribuan gambar kucing beserta label "kucing", dan ribuan gambar anjing beserta label "anjing". Model kemudian belajar memetakan fitur-fitur visual pada gambar ke label yang benar. Inti dari pendekatan ini adalah adanya "guru" yang memberikan jawaban benar untuk setiap contoh pelatihan. Kekurangan utamanya adalah ketergantungan pada ketersediaan data berlabel yang berkualitas dan dalam jumlah besar, yang seringkali mahal dan memakan waktu untuk diperoleh.
- **Unsupervised Learning (Pembelajaran Tak Terawasi):** Dalam *unsupervised learning*, model bekerja dengan data yang tidak memiliki label. Tujuannya adalah menemukan pola, struktur, atau pengelompokan tersembunyi di dalam data. Contohnya adalah *clustering* (pengelompokan) pelanggan berdasarkan perilaku belanja mereka, atau reduksi dimensi untuk memvisualisasikan data berdimensi tinggi. Tidak ada umpan balik "benar atau salah" yang diterima oleh model.
- **Reinforcement Learning (Pembelajaran Penguatan):** RL berada di antara kedua paradigma di atas dalam hal umpan balik yang diterima. Tidak seperti *supervised learning*, RL tidak mendapatkan jawaban "benar" *agent* tidak diberitahu tindakan mana yang seharusnya diambil. Namun, tidak seperti *unsupervised learning*, RL mendapatkan umpan balik berupa sinyal *reward* yang menunjukkan seberapa baik atau buruk tindakan yang diambil. Umpan balik ini bersifat evaluatif (menilai seberapa baik tindakan yang diambil) bukan instruktif (memberitahu tindakan yang benar). Selain itu, umpan balik dalam RL seringkali bersifat tertunda (*delayed*) konsekuensi dari suatu tindakan mungkin baru terasa setelah banyak langkah ke depan, yang membuat masalah *credit assignment* (menentukan tindakan mana yang berkontribusi pada kesuksesan atau kegagalan) menjadi tantangan tersendiri [1].

Terdapat beberapa karakteristik unik yang membedakan masalah RL dari masalah *machine learning* lainnya [1][5]: (1) Tidak ada pengawas (*supervisor*) yang memberitahu jawaban benar, hanya sinyal *reward*; (2) Umpan balik bersifat tertunda, bukan langsung; (3) Waktu

(*time*) dan urutan (*sequence*) menjadi faktor penting; (4) Tindakan *agent* mempengaruhi data yang diterima selanjutnya, menciptakan ketergantungan antara keputusan dan observasi.

2.2 Komponen Utama Reinforcement Learning

Sistem *reinforcement learning* terdiri dari beberapa komponen utama yang saling berinteraksi [1][5]:

2.2.1 Agent (Agen)

Agent adalah entitas yang bertindak sebagai pengambil keputusan (*decision-maker*) atau pembelajar (*learner*) dalam sistem RL [1]. *Agent* bertanggung jawab untuk mengamati kondisi lingkungan saat ini, memilih tindakan yang akan diambil berdasarkan kebijakan (*policy*) yang dimilikinya, dan kemudian menerima umpan balik berupa *reward* dari lingkungan sebagai konsekuensi dari tindakan tersebut.

Sebagai analogi, bayangkan seorang pengemudi taksi di kota yang tidak dikenalnya. Pengemudi tersebut adalah *agent* ia harus membuat keputusan di setiap persimpangan: belok kiri, belok kanan, atau lurus. Ia mengamati lingkungan sekitarnya (peta, rambu lalu lintas, kondisi jalan), mengambil keputusan berdasarkan strategi atau intuisinya, dan menerima umpan balik semakin cepat ia sampai ke tujuan, semakin besar tip yang diterimanya (*reward* positif); jika ia tersesat dan membuang waktu, ia mendapat keluhan dari penumpang (*reward* negatif).

Tujuan akhir *agent* adalah mempelajari kebijakan optimal (*optimal policy*) yang memaksimalkan *reward* kumulatif dalam jangka panjang. Penting untuk ditekankan bahwa *agent* tidak hanya mengejar *reward* sesaat, tetapi memikirkan dampak jangka panjang dari setiap tindakannya. Misalnya, dalam catur, mengorbankan pion sekarang (*reward* negatif sesaat) mungkin mengarah pada kemenangan di akhir permainan (*reward* positif besar di masa depan) [1].

2.2.2 Environment (Lingkungan)

Environment merepresentasikan segala sesuatu di luar *agent* yang berinteraksi dengannya [1]. Lingkungan mendefinisikan aturan, dinamika, serta "dunia" tempat *agent* beroperasi. Dalam konteks formal, lingkungan menerima tindakan dari *agent* pada setiap langkah waktu, kemudian merespons dengan dua hal: (1) memberikan *state* (keadaan) baru yang merepresentasikan kondisi lingkungan setelah tindakan diambil, dan (2) memberikan sinyal *reward* numerik yang mengevaluasi tindakan tersebut.

Lingkungan dapat bersifat deterministik (tindakan yang sama di situasi yang sama selalu menghasilkan hasil yang sama) atau stokastik (terdapat unsur keacakan dalam hasil tindakan). Sebagai contoh, permainan catur adalah lingkungan deterministik langkah yang sama selalu menghasilkan posisi papan yang sama. Sebaliknya, permainan dadu adalah lingkungan stokastik hasil lemparan dadu bersifat acak meskipun pemain melakukan tindakan yang sama.

Selain itu, lingkungan juga dapat dibedakan berdasarkan observabilitas-nya [1]. Dalam lingkungan yang *fully observable*, *agent* dapat mengamati seluruh keadaan lingkungan secara lengkap (misalnya, permainan catur di mana seluruh papan terlihat). Dalam lingkungan yang *partially observable*, *agent* hanya dapat melihat sebagian dari keadaan lingkungan (misalnya, permainan poker di mana pemain tidak dapat melihat kartu lawan). Lingkungan *partially observable* jauh lebih menantang karena *agent* harus membuat keputusan berdasarkan informasi yang tidak lengkap.

2.2.3 State (Keadaan)

State merepresentasikan kondisi atau situasi saat ini dari lingkungan yang diamati oleh *agent* pada suatu waktu tertentu [1]. Secara konseptual, *state* adalah ringkasan dari semua informasi yang diperlukan oleh *agent* untuk membuat keputusan yang optimal. Himpunan seluruh *state* yang mungkin disebut sebagai *state space* (S).

State space dapat bersifat diskret (jumlah *state* terhingga dan dapat dihitung) atau kontinu (jumlah *state* tak terhingga). Sebagai contoh:

- Diskret: Dalam permainan tic-tac-toe, setiap konfigurasi papan (posisi X dan O) merepresentasikan satu *state*. Totalnya ada sekitar 5.478 *state* yang valid.
- Kontinu: Dalam sistem kontrol robot, *state* mungkin berupa sudut dan kecepatan setiap sendi robot, yang merupakan nilai kontinu (bilangan real). Setiap kombinasi sudut dan kecepatan yang sedikit berbeda merupakan *state* yang berbeda, sehingga jumlah *state*-nya tak terhingga.

Ukuran dan sifat *state space* sangat mempengaruhi pemilihan algoritma RL yang tepat. Algoritma berbasis tabel (seperti Q-Learning) hanya praktis untuk *state space* yang kecil dan diskret, sementara *state space* yang besar atau kontinu memerlukan teknik aproksimasi fungsi seperti *deep neural network* [10].

2.2.4 Action (Tindakan)

Action adalah tindakan yang diambil oleh *agent* pada suatu *state* tertentu [1]. Setelah mengamati *state*, *agent* memilih salah satu tindakan dari himpunan tindakan yang tersedia, yang disebut *action space* (A). Tindakan ini akan mempengaruhi dinamika lingkungan dan menentukan *state* selanjutnya serta *reward* yang diterima.

Serupa dengan *state space*, *action space* juga dapat bersifat diskret atau kontinu:

- Diskret: Dalam permainan video, *action* mungkin berupa {atas, bawah, kiri, kanan, tembak}. *Agent* memilih satu dari beberapa pilihan terbatas.
- Kontinu: Dalam pengendalian robot, *action* mungkin berupa besarnya torsi yang diterapkan pada sebuah motor, yang merupakan nilai kontinu. *Agent* harus memilih nilai numerik yang tepat, bukan sekadar memilih dari beberapa pilihan.

Perlu dicatat bahwa *action space* bisa berbeda-beda tergantung pada *state* saat ini. Misalnya, dalam permainan kartu, tindakan yang tersedia bergantung pada kartu yang dimiliki pemain saat itu.

2.2.5 Reward (Imbalan)

Reward adalah sinyal numerik skalar (satu angka) yang diberikan oleh lingkungan kepada *agent* setelah *agent* mengambil suatu tindakan [1]. *Reward* merupakan mekanisme umpan balik utama dalam RL dan berfungsi sebagai "kompas" yang mengarahkan pembelajaran *agent*. Tanpa sinyal *reward*, *agent* tidak memiliki cara untuk mengetahui apakah tindakannya baik atau buruk. *Reward* bersifat:

- Positif: Menandakan tindakan yang menguntungkan dan diinginkan. Contoh: dalam permainan, mendapatkan poin; dalam robotik, robot berhasil menggenggam objek.
- Negatif (hukuman/penalty): Menandakan tindakan yang merugikan dan harus dihindari. Contoh: dalam navigasi, menabrak dinding; dalam permainan, kehilangan nyawa.
- Nol: Menandakan tindakan yang netral tidak secara langsung menguntungkan maupun merugikan.

Perancangan fungsi *reward* (*reward shaping*) merupakan salah satu aspek paling krusial dan menantang dalam penerapan RL [1]. Fungsi *reward* yang dirancang dengan buruk dapat

mengarahkan *agent* untuk belajar perilaku yang tidak diinginkan. Sebagai contoh klasik, jika sebuah robot pembersih hanya diberi *reward* setiap kali ia memungut sampah, ia mungkin akan belajar untuk menyebarkan sampah terlebih dahulu agar bisa memungutnya lagi secara teknis memaksimalkan *reward*, tetapi mengalahkan tujuan sebenarnya. Fenomena ini dikenal sebagai *reward hacking* dan menunjukkan pentingnya merancang *reward function* yang benar benar mencerminkan tujuan yang diinginkan.

Tujuan utama *agent* bukan memaksimalkan *reward* sesaat pada satu langkah, melainkan memaksimalkan return (pengembalian), yaitu akumulasi *reward* yang didiskon (*discounted*) sepanjang waktu [1]:

$$G_t = R_{\{t+1\}} + \gamma R_{\{t+2\}} + \gamma^2 R_{\{t+3\}} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{\{t+k+1\}}$$

di mana $\gamma \in [0, 1]$ adalah discount factor (faktor diskon) yang menentukan seberapa penting *reward* di masa depan dibandingkan *reward* saat ini. Pemahaman tentang *discount factor* sangat penting:

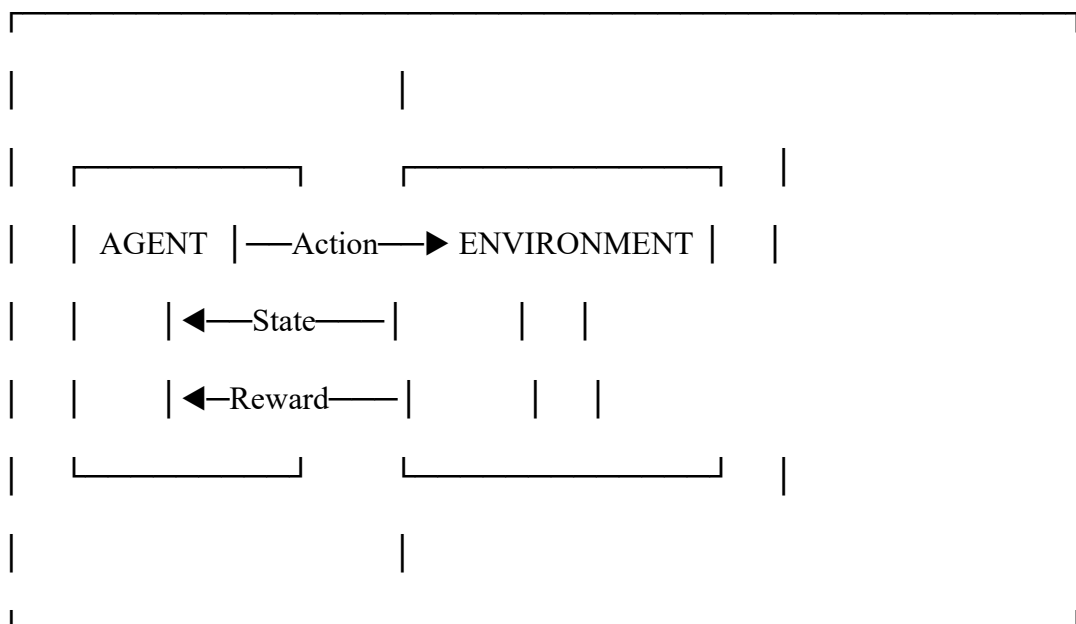
- $\gamma = 0$ (*myopic agent*): *Agent* hanya mementingkan *reward* langsung/sesaat dan sama sekali mengabaikan *reward* di masa depan. Ini menghasilkan perilaku "*greedy*" yang hanya berpikir jangka pendek.
- $\gamma = 1$ (*far-sighted agent*): *Agent* menghargai *reward* masa depan sama besarnya dengan *reward* saat ini. Ini membuat *agent* berpikir sangat jauh ke depan, tetapi dapat menyebabkan masalah konvergensi pada tugas yang tidak memiliki titik akhir (*continuing tasks*).
- $0 < \gamma < 1$: Nilai yang umum digunakan (misalnya $\gamma = 0.9$ atau $\gamma = 0.99$). *Agent* mempertimbangkan *reward* masa depan tetapi memberikan bobot lebih besar pada *reward* yang lebih dekat secara temporal. Semakin tinggi nilai γ , semakin jauh ke depan *agent* "berpikir".

Sebagai ilustrasi, misalkan $\gamma = 0.9$ dan *agent* menerima *reward* 10 di setiap langkah ke depan. Maka kontribusi *reward* dari langkah ke-1 adalah 10, dari langkah ke-2 adalah $10 \times 0.9 = 9$, dari langkah ke-3 adalah $10 \times 0.81 = 8.1$, dan seterusnya. *Reward* yang jauh di masa depan berkontribusi semakin kecil, yang secara intuitif masuk akal kita biasanya menilai imbalan yang lebih pasti (dekat) lebih tinggi daripada imbalan yang tidak pasti (jauh).

2.2.6 Siklus Interaksi Agent-Environment

Interaksi antara *agent* dan *environment* dalam RL mengikuti siklus berulang yang merupakan inti dari keseluruhan proses pembelajaran [1]. Siklus ini berlangsung dalam langkah-langkah waktu diskret (*discrete time steps*) $t = 0, 1, 2, 3, \dots$ sebagai berikut:

1. Pada langkah waktu t , *agent* mengamati *state* saat ini S_t dari lingkungan.
2. Berdasarkan *state* S_t dan *policy* yang dimilikinya, *agent* memilih *action* A_t .
3. *Agent* mengeksekusi *action* A_t pada lingkungan.
4. Lingkungan merespons dengan berpindah ke *state* baru $S_{\{t+1\}}$ dan memberikan *reward* $R_{\{t+1\}}$.
5. *Agent* menggunakan informasi transisi $(S_t, A_t, R_{\{t+1\}}, S_{\{t+1\}})$ untuk memperbarui kebijakannya yaitu memperbaiki strategi pemilihan tindakan agar lebih baik di masa depan.
6. Proses berulang dari langkah 1 dengan *state* baru $S_{\{t+1\}}$.



Rangkaian interaksi dari awal hingga akhir (mencapai *state* terminal) disebut episode atau trajectory. Contoh satu episode adalah satu permainan catur lengkap dari langkah pertama hingga *checkmate* atau *draw*. Dalam RL, *agent* biasanya belajar dari banyak episode semakin banyak episode yang dialami, semakin baik *policy* yang dipelajari karena *agent* telah mengalami lebih banyak situasi berbeda.

Tidak semua tugas RL memiliki *state* terminal. Tugas yang memiliki titik akhir disebut episodic tasks (misalnya, permainan yang berakhir dengan menang/kalah). Tugas yang berlangsung terus-menerus tanpa titik akhir alami disebut continuing tasks (misalnya, pengelolaan portofolio investasi yang berlangsung tanpa batas waktu) [1].

2.3 Markov Decision Process (MDP)

Markov Decision Process (MDP) merupakan kerangka matematis formal yang digunakan untuk memodelkan masalah pengambilan keputusan sekuensial dalam *reinforcement learning* [1][6]. MDP menyediakan fondasi teoretis yang kuat dan elegan untuk merumuskan interaksi antara *agent* dan *environment*. Hampir seluruh masalah RL dapat diformulasikan sebagai MDP, yang menjadikannya bahasa universal untuk mendiskusikan dan menganalisis algoritma-algoritma RL.

Konsep MDP pertama kali dikembangkan oleh Richard Bellman pada tahun 1950-an dalam konteks pemrograman dinamis (*dynamic programming*) [6]. Bellman menunjukkan bahwa masalah pengambilan keputusan yang kompleks dapat dipecah menjadi sub-masalah yang lebih sederhana dan diselesaikan secara rekursif sebuah prinsip yang kemudian menjadi fondasi bagi banyak algoritma RL.

2.3.1 Definisi Formal MDP

Sebuah MDP secara formal didefinisikan oleh *tuple* (5-tuple) (S, A, P, R, γ) di mana [1][6]:

- **S** — *State space*: Himpunan seluruh *state* yang mungkin dalam lingkungan. Ini merupakan deskripsi lengkap dari semua situasi yang mungkin dihadapi oleh *agent*.
- **A** — *Action space*: Himpunan seluruh *action* yang mungkin dilakukan oleh *agent*. Pada beberapa formulasi, *action space* bisa bergantung pada *state*, dinotasikan sebagai $A(s)$.
- **$P(s'|s, a)$** — *Transition probability function* (fungsi probabilitas transisi): Mendefinisikan probabilitas berpindah ke *state* s' jika *agent* mengambil *action* a di *state* s . Fungsi ini merangkum keseluruhan dinamika lingkungan bagaimana lingkungan berubah sebagai respons terhadap tindakan *agent*. Secara formal: $P(s'|s, a) = \Pr(S_{t+1} = s' | S_t = s, A_t = a)$. Jumlah seluruh probabilitas transisi dari satu *state-action* harus sama dengan 1: $\sum_{s'} P(s'|s, a) = 1$ untuk semua s dan a .

- **$R(s, a, s')$** — *Reward function* (fungsi imbalan): Mendefinisikan *reward* numerik yang diterima *agent* setelah melakukan transisi dari *state* s ke *state* s' melalui *action* a . Pada beberapa formulasi, *reward function* disederhanakan menjadi $R(s, a)$ atau bahkan $R(s)$.
- **γ** — *Discount factor* (faktor diskon): Nilai dalam rentang $[0, 1]$ yang mengontrol seberapa besar pengaruh *reward* masa depan relatif terhadap *reward* saat ini, sebagaimana telah dijelaskan di Sub Bab 2.2.5.

Untuk memberikan contoh konkret, perhatikan sebuah *grid world* sederhana berukuran 3×3 di mana sebuah robot harus mencapai posisi tujuan:

- **S** = $\{(1,1), (1,2), (1,3), (2,1), \dots, (3,3)\}$ 9 posisi pada *grid*
- **A** = {Atas, Bawah, Kiri, Kanan} 4 gerakan
- **P** : Jika robot di posisi $(1,1)$ dan memilih "Kanan", maka $P((1,2) | (1,1), \text{Kanan}) = 1.0$ (deterministik). Pada lingkungan stokastik, mungkin $P((1,2) | (1,1), \text{Kanan}) = 0.8$ (berhasil bergerak kanan) dan $P((1,1) | (1,1), \text{Kanan}) = 0.2$ (tetap di tempat karena tergelincir).
- **R** : $R = -1$ untuk setiap langkah (mendorong robot bergerak efisien), $R = +10$ untuk mencapai tujuan.
- **γ** = 0.9

2.3.2 Markov Property (Sifat Markov)

Karakteristik fundamental dan esensial dari MDP adalah sifat Markov (*Markov property*), yang dinamai dari matematikawan Rusia Andrey Markov. Sifat ini menyatakan bahwa *state* masa depan hanya bergantung pada *state* dan *action* saat ini, bukan pada sejarah *state* dan *action* sebelumnya [1][6]:

$$P(S_{t+1} | S_t, A_t, S_{t-1}, A_{t-1}, \dots, S_0, A_0) = P(S_{t+1} | S_t, A_t)$$

Dengan kata lain, *state* saat ini telah mengandung semua informasi dari masa lalu yang relevan untuk membuat keputusan di masa depan. Masa lalu menjadi "tidak relevan" setelah kita mengetahui keadaan saat ini. Ini merupakan asumsi yang sangat kuat dan menyederhanakan masalah secara signifikan *agent* hanya perlu mempertimbangkan kondisi saat ini untuk membuat keputusan, tanpa perlu menyimpan dan mengingat seluruh riwayat interaksi.

Sebagai analogi, perhatikan permainan catur. Jika kita mengetahui posisi semua bidak di papan saat ini (termasuk informasi tentang apakah raja sudah pernah bergerak, untuk aturan *castling*), kita memiliki semua informasi yang diperlukan untuk menentukan langkah terbaik. Tidak masalah *bagaimana* posisi tersebut tercapai apakah melalui pembukaan Sicilian atau pembukaan Ruy Lopez. Yang penting hanyalah konfigurasi papan saat ini. Inilah esensi dari sifat Markov.

Namun, penting untuk dicatat bahwa sifat Markov tidak selalu terpenuhi secara sempurna di dunia nyata. Banyak masalah di mana *state* yang diamati tidak mengandung semua informasi relevan misalnya, dalam permainan poker di mana *agent* tidak dapat melihat kartu lawan. Masalah semacam ini dimodelkan sebagai *Partially Observable MDP (POMDP)*, yang merupakan perluasan dari MDP standar [1].

2.4 Policy dan Value Function

Dua konsep yang paling sentral dalam teori RL adalah *policy* (kebijakan) dan *value function* (fungsi nilai). *Policy* mendefinisikan "strategi" *agent* dalam bertindak, sementara *value function* memberikan cara untuk mengevaluasi seberapa baik strategi tersebut. Hubungan antara keduanya sangat erat: *agent* menggunakan *value function* untuk mengevaluasi dan memperbaiki *policy*-nya secara iteratif [1].

2.4.1 Policy (Kebijakan) – π

Policy (π) merupakan inti dari perilaku *agent* ia mendefinisikan strategi atau aturan yang digunakan *agent* untuk memilih tindakan berdasarkan *state* yang diamati [1]. *Policy* adalah pemetaan dari *state* ke *action* (atau distribusi probabilitas atas *action*). Dengan kata lain, *policy* menjawab pertanyaan: "Jika saya berada di situasi ini, apa yang harus saya lakukan?"

Policy dapat bersifat:

- **Deterministik:** $\pi(s) = a$ Untuk setiap state, policy memberikan satu action pasti yang harus diambil. Tidak ada unsur keacakan. Contoh deterministik: "Jika lampu merah, berhenti. Jika lampu hijau, jalan."
- **Stokastik:** $\pi(a|s) = P(A_t = a | S_t = s)$ Policy memberikan distribusi probabilitas atas semua action yang mungkin di setiap state. Contoh stokastik: "Jika di persimpangan, 70% kemungkinan belok kanan, 30% kemungkinan lurus." Policy stokastik berguna dalam situasi di mana deterministik bisa diprediksi

oleh lawan (seperti dalam permainan kompetitif), atau ketika agent masih perlu melakukan eksplorasi.

Sebuah *policy* bisa sangat sederhana (misalnya, "selalu belok kiri") atau sangat kompleks (misalnya, jaringan neural yang memproses jutaan parameter untuk menentukan tindakan). Tujuan utama dari seluruh proses RL adalah menemukan *policy* optimal π^* yaitu *policy* yang memaksimalkan *expected return* (akumulasi *reward* yang diharapkan) dari setiap *state* [1]. *Policy* optimal menjamin bahwa tidak ada *policy* lain yang dapat menghasilkan *return* lebih tinggi.

2.4.2 State-Value Function – $V^{\pi(s)}$

Untuk mengevaluasi seberapa baik suatu *policy*, kita memerlukan cara untuk mengukur "nilai" dari setiap *state* di bawah *policy* tersebut. Inilah fungsi dari state-value function $V^{\pi(s)}$ [1].

State-value function $V^{\pi(s)}$ mengestimasi seberapa baik (*how good*) bagi *agent* untuk berada di *state* s , dengan asumsi ia mengikuti *policy* π mulai dari *state* tersebut dan seterusnya:

$$V^{\pi(s)} = E_{\pi}[G_t | S_t = s] = E_{\pi}[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s]$$

Fungsi ini memberikan ekspektasi *return* (akumulasi *reward* terdiskon) yang akan diperoleh jika *agent* memulai dari *state* s dan mengikuti *policy* π selanjutnya. "Ekspektasi" digunakan karena lingkungan mungkin bersifat stokastik dan *policy* mungkin juga stokastik, sehingga *return* yang diperoleh bisa bervariasi di setiap percobaan.

Sebagai contoh sederhana, dalam sebuah *maze* (labirin), *state* yang berdekatan dengan pintu keluar akan memiliki nilai $V^{\pi(s)}$ yang tinggi (karena hanya perlu beberapa langkah lagi untuk mencapai tujuan dan mendapatkan *reward*), sementara *state* yang jauh dari pintu keluar atau berada di jalan buntu akan memiliki nilai yang rendah. Dengan mengetahui $V^{\pi(s)}$ untuk setiap *state*, *agent* dapat "melihat" peta nilai dari seluruh lingkungan dan mengetahui *state* mana yang menguntungkan dan mana yang merugikan.

2.4.3 Action-Value Function (Q-Function) – $Q^{\pi(s,a)}$

Selain *state-value function*, terdapat fungsi nilai yang bahkan lebih berguna secara praktis: action-value function atau yang sering disebut Q-function [1].

Q -function $Q^{\pi(s,a)}$ mengestimasi seberapa baik mengambil suatu *action* tertentu a pada *state* s , dan kemudian mengikuti *policy* π untuk selanjutnya:

$$Q^{\pi(s,a)} = E_{\pi} [G_t | S_t = s, A_t = a]$$

Perbedaan kunci antara $V^{\pi(s)}$ dan $Q^{\pi(s,a)}$ adalah: $V^{\pi(s)}$ menilai seberapa baik sebuah *state* secara keseluruhan (dengan mempertimbangkan *policy* yang digunakan untuk memilih *action*), sedangkan $Q^{\pi(s,a)}$ menilai seberapa baik mengambil *action* spesifik a di *state* s . Ini sangat berguna karena *agent* dapat langsung membandingkan nilai dari setiap *action* yang tersedia dan memilih yang terbaik.

Sebagai ilustrasi, bayangkan Anda sedang di sebuah persimpangan jalan (*state* s). $V^{\pi(s)}$ memberi tahu Anda seberapa baik persimpangan tersebut secara umum. Sementara $Q^{\pi(s, \text{"belok kiri"})}$ memberi tahu Anda seberapa baik jika Anda memilih belok kiri, $Q^{\pi(s, \text{"belok kanan"})}$ memberi tahu seberapa baik belok kanan, dan $Q^{\pi(s, \text{"lurus"})}$ memberi tahu seberapa baik jalan lurus. Dengan membandingkan ketiga nilai Q ini, Anda bisa langsung memilih arah yang terbaik tanpa perlu informasi tambahan.

Q -function menjadi sangat penting dalam banyak algoritma RL khususnya Q-Learning dan DQN karena *policy* optimal dapat langsung diturunkan dari Q -function optimal: cukup pilih *action* dengan nilai Q tertinggi di setiap *state* [1].

Hubungan antara keduanya dapat dinyatakan sebagai:

$$V^{\pi(s)} = \sum_a \pi(a|s) Q^{\pi(s,a)}$$

Artinya, nilai sebuah *state* equals to penjumlahan dari nilai setiap *action* yang dikalikan probabilitas memilih *action* tersebut menurut *policy* π .

2.4.4 Persamaan Bellman

Salah satu hasil terpenting dalam teori RL adalah Persamaan Bellman (*Bellman Equation*), yang diformulasikan oleh Richard Bellman [6]. Persamaan ini menyatakan hubungan rekursif antara nilai sebuah *state* (atau *state-action*) dengan nilai *state-state* penerusnya. Sifat rekursif ini sangat penting karena memungkinkan *value function* dihitung secara iteratif kita tidak perlu menghitung seluruh *return* dari awal hingga akhir, melainkan cukup melihat satu langkah ke depan [1].

Untuk State-Value (Bellman Expectation Equation):

$$V^{\pi(s)} = \sum_a \pi(a|s) \sum_{\{s'\}} P(s'|s, a) [R(s, a, s') + \gamma V^{\pi(s')}]$$

Persamaan ini dapat dibaca sebagai: "Nilai *state* s di bawah *policy* π sama dengan rata-rata terbobot dari *reward* langsung yang diterima ditambah nilai *state* berikutnya yang didiskon, di mana bobot ditentukan oleh probabilitas *action* (dari *policy*) dan probabilitas transisi (dari lingkungan)."

Untuk Action-Value:

$$Q^{\pi(s,a)} = \sum_{\{s'\}} P(s'|s, a) \left[R(s, a, s') + \gamma \sum_{\{a'\}} \pi(a'|s') Q \right]$$

Selain Persamaan Bellman Ekspektasi di atas, terdapat juga Bellman Optimality Equation yang mendefinisikan *value function* optimal *value function* yang dihasilkan oleh *policy* optimal [1]:

$$\begin{aligned} V(s) &= \max_a \sum_{\{s'\}} P(s'|s, a) [R(s, a, s') + \gamma V(s')] ** Q(s, a) \\ &= \sum_{\{s'\}} P(s'|s, a) [R(s, a, s') + \gamma \max_{\{a'\}} Q(s', a')] ** \end{aligned}$$

Persamaan Bellman Optimalitas menyatakan bahwa nilai optimal dari suatu *state* sama dengan *return* yang diharapkan jika *agent* mengambil tindakan terbaik di setiap langkah. Persamaan ini menjadi fondasi langsung bagi algoritma Q-Learning, di mana operator *max* digunakan untuk memperbarui nilai Q.

Pentingnya Persamaan Bellman tidak bisa dilebih-lebihkan ia menjadi dasar bagi hampir seluruh algoritma RL, baik yang klasik (seperti *dynamic programming*, Q-Learning, SARSA) maupun yang modern (seperti DQN dan varian-variananya). Tanpa hubungan rekursif ini, menghitung *value function* akan membutuhkan *forward simulation* dari setiap *state* hingga akhir episode, yang sangat tidak efisien [1][6].

2.5 Eksplorasi vs. Eksploitasi

Salah satu tantangan paling fundamental dan unik dalam RL yang tidak ditemukan di paradigma *machine learning* lainnya adalah dilema eksplorasi vs. eksploitasi (*exploration vs. exploitation dilemma*) [1]:

- **Eksploitasi (Exploitation):** *Agent* memanfaatkan pengetahuan yang sudah dimilikinya dan memilih *action* yang sudah diketahui memberikan *reward* tertinggi

berdasarkan estimasi saat ini. Eksploitasi memaksimalkan *reward* jangka pendek berdasarkan informasi yang tersedia.

- **Eksplorasi** (*Exploration*): *Agent* sengaja mencoba *action* baru yang belum pernah atau jarang dicoba, dengan harapan menemukan tindakan yang mungkin lebih baik dari yang sudah diketahui. Eksplorasi mengorbankan *reward* jangka pendek demi potensi penemuan strategi yang lebih baik.

Dilema ini mudah dipahami melalui analogi restoran: Anda memiliki restoran favorit yang selalu Anda kunjungi dan Anda tahu makanannya enak (eksploitasi). Namun, ada banyak restoran baru di kota yang belum pernah Anda coba. Salah satu dari restoran baru itu mungkin jauh lebih enak dari favorit Anda, tetapi Anda tidak akan pernah tahu jika Anda tidak pernah mencobanya (eksplorasi). Di sisi lain, mencoba restoran baru berisiko makanannya mungkin mengecewakan. Bagaimana Anda menyeimbangkan antara kenyamanan restoran favorit dan potensi penemuan restoran yang lebih baik?

Keseimbangan antara eksplorasi dan eksploitasi sangat krusial karena [1]:

- Jika *agent* **terlalu banyak mengeksploitasi** (*too greedy*), ia mungkin terjebak pada kebijakan *sub-optimal*. Ia mungkin menemukan tindakan yang "cukup baik" di awal dan terus menggunakannya, tanpa pernah menemukan tindakan yang sebenarnya jauh lebih baik.
- Jika *agent* terlalu banyak mengeksplorasi, ia membuang-buang waktu dan sumber daya mencoba tindakan yang tidak berguna dan sudah terbukti buruk, tanpa pernah memanfaatkan pengetahuan yang sudah diperoleh.

Beberapa strategi yang umum digunakan untuk menyeimbangkan eksplorasi dan eksploitasi adalah [1]:

- **ϵ -Greedy** (Epsilon-Greedy): Ini adalah strategi yang paling sederhana dan paling banyak digunakan. Dengan probabilitas ϵ (misalnya 0.1 atau 10%), *agent* memilih *action* secara acak (eksplorasi); dengan probabilitas $1 - \epsilon$ (misalnya 0.9 atau 90%), *agent* memilih *action* dengan nilai Q tertinggi (eksploitasi). Sering kali, nilai ϵ diturunkan secara bertahap selama pelatihan dimulai dari nilai tinggi (misalnya 1.0, eksplorasi penuh) dan perlahan diturunkan mendekati 0 seiring *agent* menjadi lebih yakin dengan estimasi nilainya. Pendekatan ini disebut ϵ -decay atau annealing.

- **Softmax/Boltzmann Exploration:** Alih-alih memilih secara acak sepenuhnya saat mengeksplorasi (seperti ϵ -greedy), metode Softmax memilih *action* berdasarkan distribusi probabilitas yang proporsional terhadap estimasi nilai Q-nya. *Action* dengan nilai Q lebih tinggi memiliki probabilitas lebih besar untuk dipilih, tetapi *action* dengan nilai Q rendah masih memiliki peluang untuk dipilih. Parameter suhu (*temperature*) τ mengontrol tingkat eksplorasi: suhu tinggi $\rightarrow$ distribusi merata (eksplorasi lebih); suhu rendah $\rightarrow$ distribusi tajam (eksploitasi lebih).
- **Upper Confidence Bound (UCB):** Strategi ini menambahkan "bonus eksplorasi" pada estimasi nilai setiap *action*. *Action* yang jarang dicoba mendapat bonus lebih besar, mendorong *agent* untuk mencoba *action* yang kurang tereksplor. Secara matematis, UCB memilih *action* yang memaksimalkan $Q(s, a) + c \times \sqrt{\frac{\ln(t)}{N(s, a)}}$, di mana $N(s, a)$ adalah jumlah kali *action* a dipilih di *state* s , dan c adalah konstanta yang mengontrol tingkat eksplorasi. Keunggulan UCB adalah ia secara otomatis mengurangi eksplorasi untuk *action* yang sudah banyak dicoba.

Pemilihan strategi eksplorasi yang tepat memiliki dampak signifikan terhadap kecepatan dan kualitas pembelajaran *agent*. Dalam praktik, ϵ -greedy dengan *decay* sering menjadi pilihan pertama karena kesederhanaannya, meskipun strategi lain mungkin lebih efisien dalam situasi tertentu [1].

2.6 Posisi Reinforcement Learning dalam Machine Learning

Untuk memberikan gambaran yang lebih jelas, berikut adalah perbandingan antara tiga paradigma utama *machine learning* [1]:

Aspek	Supervised Learning	Unsupervised Learning	Reinforcement Learning
Data	Data berlabel (input-output)	Data tidak berlabel	Tidak ada dataset; interaksi langsung
Umpan Balik	Label/jawaban benar	Tidak ada umpan balik	Sinyal <i>reward</i> (tertunda)
Tujuan	Memprediksi output	Menemukan pola/struktur	Memaksimalkan <i>reward</i> kumulatif
Proses Belajar	Dari contoh	Dari pola data	Dari coba-coba (<i>trial-and-error</i>)
Contoh Aplikasi	Klasifikasi gambar, NLP	Clustering, reduksi dimensi	ChatGPT (RLHF), sistem rekomendasi, robotik

BAB III

LANGKAH-LANGKAH DAN ALGORITMA REINFORCEMENT LEARNING

3.1 Kerangka Umum Algoritma Reinforcement Learning

Sebelum membahas algoritma-algoritma spesifik, penting untuk memahami kerangka kerja umum yang diikuti dalam menerapkan dan mengembangkan solusi *reinforcement learning*. Kerangka ini berlaku untuk hampir semua algoritma RL, meskipun detail implementasinya bervariasi [1][5]:

1. **Pendefinisian Masalah sebagai MDP:** Langkah pertama dan terpenting adalah memformulasikan masalah dunia nyata ke dalam kerangka MDP. Ini mencakup mendefinisikan *state space* (apa saja informasi yang diamati oleh *agent*?), *action space* (apa saja tindakan yang tersedia?), dan *reward function* (bagaimana cara mengukur keberhasilan *agent*?). Langkah ini memerlukan pemahaman domain yang mendalam dan seringkali menjadi faktor penentu keberhasilan penerapan RL. Definisi *state* yang terlalu sederhana membuat *agent* kekurangan informasi, sementara *state* yang terlalu kompleks membuat pembelajaran menjadi lambat.
2. **Pemilihan Algoritma:** Berdasarkan karakteristik masalah, pilihlah algoritma RL yang paling sesuai. Pertimbangkan ukuran *state space* (kecil → tabel/tabular methods; besar → neural network), sifat *action space* (diskret → Q-Learning/DQN; kontinu → policy gradient), ketersediaan model lingkungan (ada → model-based; tidak ada → model-free), dan kebutuhan keamanan (perlu aman → SARSA/on-policy; fleksibel → Q-Learning/off-policy).
3. **Inisialisasi:** Menginisialisasi parameter-parameter pembelajaran. Untuk algoritma tabular (Q-Learning, SARSA), ini berarti membuat *Q-table* dan mengisinya dengan nilai awal (biasanya 0 atau nilai acak kecil). Untuk algoritma berbasis neural network (DQN), ini berarti menginisialisasi bobot jaringan neural. Nilai inisialisasi dapat mempengaruhi kecepatan konvergensi inisialisasi optimis (nilai awal tinggi) dapat mendorong eksplorasi di awal pelatihan.
4. **Proses Pembelajaran (Training Loop):** Ini adalah tahap inti di mana *agent* belajar melalui interaksi berulang dengan *environment*:
 - *Agent* mengobservasi *state* saat ini.

- *Agent* memilih *action* berdasarkan *policy* saat ini (dengan strategi eksplorasi).
 - *Agent* mengeksekusi *action* dan menerima *reward* serta *state* baru.
 - *Agent* menggunakan informasi transisi $(S_t, A_t, R_{t+1}, S_{t+1})$ untuk memperbarui parameter modelnya (Q-table atau bobot neural network).
 - Proses ini diulangi selama ribuan hingga jutaan *episode* atau *step* hingga *policy* konvergen atau mencapai performa yang memuaskan.
5. **Evaluasi dan Pengujian:** Setelah pelatihan, evaluasi *policy* yang telah dipelajari dengan menjalankannya pada *environment* tanpa eksplorasi ($\epsilon = 0$) untuk mengukur performa sebenarnya. Bandingkan dengan *baseline* yang relevan (misalnya, *policy* acak, metode heuristik, atau performa manusia).
 6. **Penyempurnaan (*Hyperparameter Tuning*):** Berdasarkan hasil evaluasi, sesuaikan *hyperparameter* seperti *learning rate* (α), *discount factor* (γ), *exploration rate* (ϵ), ukuran *replay buffer*, arsitektur neural network, dll. Proses ini bersifat iteratif dan seringkali memerlukan banyak eksperimen.

Algoritma RL sendiri dapat diklasifikasikan ke dalam beberapa kategori berdasarkan pendekatan yang digunakan [7]:

- **Model-Based vs. Model-Free:** Algoritma model-based secara eksplisit membangun model internal dari dinamika lingkungan (mempelajari fungsi transisi P dan fungsi reward R), kemudian menggunakannya untuk merencanakan tindakan tanpa harus berinteraksi langsung dengan lingkungan. Keuntungannya adalah efisiensi data yang tinggi, tetapi kelemahannya adalah model mungkin tidak akurat. Algoritma model-free tidak membangun model lingkungan ia langsung mempelajari *value function* atau *policy* dari pengalaman interaksi. Pendekatan ini lebih sederhana dan robust terhadap kesalahan model, tetapi umumnya memerlukan lebih banyak data pelatihan. Q-Learning, SARSA, dan DQN yang akan dibahas di makalah ini semuanya termasuk algoritma *model-free*.
- **On-Policy vs. Off-Policy:** Perbedaan ini sangat penting untuk dipahami. Algoritma on-policy (contoh: SARSA) mempelajari dan mengevaluasi *policy* yang *sama* dengan yang digunakan untuk mengambil tindakan dan mengeksplorasi lingkungan. Artinya, *policy* yang dipelajari dan *policy* yang dieksekusi adalah satu dan sama. Algoritma off-policy (contoh: Q-Learning, DQN) dapat mempelajari *policy* optimal yang *berbeda* dari *policy* yang digunakan untuk

eksplorasi. Ini memberikan fleksibilitas lebih besar *agent* bisa mengeksplorasi secara agresif sambil tetap mempelajari *policy* yang optimal. Perbedaan ini akan lebih jelas terlihat ketika kita membandingkan Q-Learning dan SARSA di sub bab berikutnya.

- **Value-Based vs. Policy-Based:** Algoritma value-based (contoh: Q-Learning, DQN) mempelajari *value function* (V atau Q) terlebih dahulu, kemudian menurunkan *policy* secara implisit dari *value function* tersebut (misalnya, dengan memilih *action* dengan nilai Q tertinggi). Algoritma policy-based (contoh: REINFORCE, PPO) langsung mengoptimalkan *policy* tanpa mempelajari *value function* secara eksplisit. Terdapat juga pendekatan actor-critic yang menggabungkan keduanya. Dalam makalah ini, fokus pembahasan adalah pada algoritma *value-based*.

3.2 Algoritma Q-Learning

Q-Learning merupakan salah satu algoritma RL yang paling fundamental, paling banyak dipelajari, dan paling banyak digunakan dalam literatur RL. Algoritma ini pertama kali diperkenalkan oleh Christopher Watkins dalam disertasi doktoralnya pada tahun 1989, kemudian dianalisis secara teoritis bersama Peter Dayan pada tahun 1992 [8]. Q-Learning merupakan algoritma *model-free* dan *off-policy* yang bertujuan mempelajari *action-value function* optimal $Q^*(s, a)$ secara langsung, tanpa perlu mengetahui model lingkungan (fungsi transisi P atau fungsi reward R).

Keistimewaan Q-Learning yang membuatnya sangat populer adalah jaminan konvergensinya: dengan syarat setiap pasangan *state-action* dikunjungi secara tak terbatas dan *learning rate* memenuhi kondisi tertentu (menurun secara bertahap), Q-Learning dijamin konvergen ke *action-value function* optimal Q^* [8]. Ini berarti *agent* pasti akan menemukan *policy* terbaik jika diberi waktu yang cukup.

3.2.1 Prinsip Kerja

Q-Learning menggunakan sebuah tabel dua dimensi yang disebut Q-table untuk menyimpan nilai $Q(s, a)$ untuk setiap pasangan *state-action*. Baris tabel merepresentasikan *state*, kolom merepresentasikan *action*, dan setiap sel berisi estimasi nilai Q untuk pasangan tersebut. Pada awalnya, tabel ini diisi dengan nilai nol atau acak, dan secara bertahap diperbarui seiring *agent* berinteraksi dengan lingkungan.

Nilai Q diperbarui secara iteratif menggunakan persamaan pembaruan yang diturunkan dari Persamaan Bellman Optimalitas [8]:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{\{t+1\}} + \gamma \max_a Q(S_{\{t+1\}}, a) - Q(S_t, A_t)]$$

Mari kita bedah setiap komponen persamaan ini secara detail:

- $Q(S_t, A_t)$: Estimasi nilai Q saat ini untuk pasangan *state-action* (S_t, A_t) . Ini adalah "keyakinan" *agent* saat ini tentang seberapa baik mengambil *action* A_t di *state* S_t .
- $\alpha \in (0, 1]$ — **Learning rate** (laju pembelajaran): Mengontrol seberapa besar pembaruan yang dilakukan pada setiap langkah. Nilai α tinggi (mendekati 1) membuat *agent* lebih cepat beradaptasi terhadap informasi baru tetapi bisa menyebabkan pembaruan yang tidak stabil. Nilai α rendah (mendekati 0) membuat pembelajaran lebih stabil tetapi lebih lambat. Secara intuitif, α menentukan seberapa besar "kepercayaan" *agent* terhadap informasi baru dibandingkan estimasi lama.
- $R_{\{t+1\}}$: *Reward* yang diterima setelah mengambil *action* A_t di *state* S_t .
- $\gamma \in [0, 1]$ — **Discount factor**: Menentukan pentingnya *reward* masa depan, seperti yang telah dibahas di Sub Bab 2.2.5.
- $\max_a Q(S_{\{t+1\}}, a)$: Nilai Q terbesar di antara semua *action* yang tersedia di *state* berikutnya $S_{\{t+1\}}$. Ini merepresentasikan estimasi terbaik dari nilai *state* berikutnya.
- $[R_{\{t+1\}} + \gamma \max_a Q(S_{\{t+1\}}, a) - Q(S_t, A_t)]$: Bagian ini disebut Temporal Difference (TD) Error selisih antara "target" baru $(R_{\{t+1\}} + \gamma \max_a Q(S_{\{t+1\}}, a))$ dan estimasi saat ini $(Q(S_t, A_t))$. TD Error mengukur "kejutan" seberapa jauh estimasi saat ini dari pengalaman yang sebenarnya. Jika TD Error positif, berarti *action* tersebut lebih baik dari yang diperkirakan; jika negatif, berarti lebih buruk.

Kunci dari Q-Learning yang menjadikannya algoritma *off-policy* adalah penggunaan operator max dalam pembaruan. *Agent* selalu memperbarui berdasarkan tindakan terbaik yang *mungkin* di *state* berikutnya, terlepas dari tindakan yang sebenarnya diambil. Ini berarti *agent* dapat belajar tentang *policy* optimal sambil mengikuti *policy* eksplorasi yang berbeda (misalnya ϵ -greedy).

3.2.2 Langkah-Langkah Algoritma Q-Learning

Berikut adalah prosedur lengkap algoritma Q-Learning [1][8]:

Algoritma: Q-Learning

Input: Learning rate α , discount factor γ , exploration rate ϵ

Output: Q-table yang merepresentasikan policy optimal

1. INISIALISASI:

Inisialisasi $Q(s, a) = 0$ untuk semua $s \in S$ dan $a \in A$

2. UNTUK setiap episode:

a. Inisialisasi state awal S_0

b. UNTUK setiap langkah t dalam episode:

i. PILIH action A_t dari state S_t menggunakan

strategi ϵ -greedy:

- Dengan probabilitas ϵ : pilih action acak

- Dengan probabilitas $(1-\epsilon)$: pilih $\operatorname{argmax}_a Q(S_t, a)$

ii. EKSEKUSI action A_t

Amati reward R_{t+1} dan state baru S_{t+1}

iii. PERBARUI Q-value:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

iv. SET $S_t \leftarrow S_{t+1}$

v. JIKA S_{t+1} adalah state terminal, AKHIRI episode

3. ULANGI langkah 2 sampai konvergen

3.2.3 Contoh Ilustrasi Q-Learning

Sebagai ilustrasi sederhana, bayangkan sebuah *grid world* berukuran 4×4 di mana *agent* harus bergerak dari posisi awal ke posisi tujuan. *Agent* dapat bergerak ke empat arah: atas, bawah, kiri, dan kanan.

- **State space:** 16 posisi pada *grid* (4×4)
- **Action space:** {Atas, Bawah, Kiri, Kanan}
- **Reward:** -1 untuk setiap langkah (mendorong efisiensi), +10 untuk mencapai tujuan, -10 untuk jatuh ke lubang

Melalui iterasi Q-Learning, *agent* secara bertahap mempelajari *Q-table* yang menyimpan nilai optimal untuk setiap pasangan *state-action*, sehingga dapat menemukan jalur terpendek ke tujuan.

3.3 Algoritma SARSA

SARSA (*State-Action-Reward-State-Action*) merupakan algoritma RL *model-free* yang bersifat on-policy [1]. Nama SARSA yang merupakan akronim berasal dari urutan lima elemen yang digunakan dalam proses pembaruannya: S_t (*state saat ini*), A_t (*action yang diambil*), $R\{t + 1\}$ (*reward yang diterima*), $S\{t+1\}$ (*state berikutnya*), $A_{\{t+1\}}$ (*action yang akan diambil di state berikutnya*). Kelima elemen inilah yang dibutuhkan untuk melakukan satu kali pembaruan Q-value.

SARSA sering dipandang sebagai "saudara" Q-Learning yang lebih konservatif dan lebih aman. Meskipun keduanya memiliki tujuan yang sama mempelajari *action-value function* perbedaan fundamental antara keduanya terletak pada cara mereka memperbarui estimasi Q, yang menghasilkan perilaku pembelajaran yang sangat berbeda.

3.3.1 Prinsip Kerja

Perbedaan utama dan paling krusial antara SARSA dan Q-Learning terletak pada target yang digunakan untuk memperbarui nilai Q. Pada Q-Learning, target menggunakan *action* terbaik yang mungkin di *state* berikutnya (operator max). Pada SARSA, target menggunakan *action* yang benar-benar diambil oleh *agent* di *state* berikutnya termasuk *action* eksplorasi acak [1]:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{\{t+1\}} + \gamma Q(S_{\{t+1\}}, A_{\{t+1\}}) - Q(S_t, A_t)]$$

Perhatikan perbedaan kunci: SARSA menggunakan $Q(S_{\{t+1\}}, A_{\{t+1\}})$ nilai Q dari *action* $A_{\{t+1\}}$ yang benar-benar dipilih di state berikutnya alih-alih $\max_a Q(S_{\{t+1\}}, a)$ yang digunakan Q-Learning.

Mengapa perbedaan kecil ini begitu signifikan? Karena $A_{\{t+1\}}$ dipilih menggunakan *policy* saat ini (misalnya ϵ -greedy), maka terkadang $A_{\{t+1\}}$ adalah *action* acak yang buruk (saat eksplorasi). SARSA memperhitungkan kemungkinan *action* buruk ini dalam estimasi nilainya, yang membuat SARSA menjadi lebih konservatif dalam menilai *state-action* yang berdekatan dengan situasi berbahaya. Inilah sebabnya SARSA disebut *on-policy*: ia mengevaluasi *policy* yang benar-benar diikuti, termasuk semua keacakan eksplorasinya.

3.3.2 Langkah-Langkah Algoritma SARSA

Berikut adalah prosedur lengkap algoritma SARSA [1]. Perhatikan perbedaan utama dengan Q-Learning: pada SARSA, *action* berikutnya ($A_{\{t+1\}}$) harus dipilih terlebih dahulu sebelum pembaruan Q-value dilakukan, karena $A_{\{t+1\}}$ dibutuhkan dalam persamaan pembaruan:

Algoritma: SARSA

Input: Learning rate α , discount factor γ , exploration rate ϵ

Output: Q-table yang merepresentasikan policy

1. INISIALISASI:

Inisialisasi $Q(s, a) = 0$ untuk semua $s \in S$ dan $a \in A$

2. UNTUK setiap episode:

a. Inisialisasi state awal S_0

b. PILIH *action* A_0 dari S_0 menggunakan strategi ϵ -greedy

c. UNTUK setiap langkah t dalam episode:

i. EKSEKUSI *action* A_t

Amati reward R_{t+1} dan state baru S_{t+1}

ii. PILIH action A_{t+1} dari S_{t+1}

menggunakan strategi ϵ -greedy

iii. PERBARUI Q-value:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

iv. SET $S_t \leftarrow S_{t+1}, A_t \leftarrow A_{t+1}$

v. JIKA S_{t+1} adalah state terminal, AKHIRI episode

3. ULANGI langkah 2 sampai konvergen

3.3.3 Perbandingan Perilaku SARSA dan Q-Learning

Perbedaan antara SARSA dan Q-Learning paling terlihat jelas dalam skenario Cliff Walking sebuah contoh klasik dari Sutton dan Barto [1]. Dalam skenario ini, *agent* harus bergerak dari titik awal ke titik tujuan melalui *grid*. Di sepanjang tepi bawah *grid* terdapat "jurang" (*cliff*) *state* yang memberikan *reward* sangat negatif (-100) dan mengembalikan *agent* ke titik awal. Jalur optimal (terpendek) melewati tepian jurang.

Hasil yang diamati:

- **Q-Learning** menemukan jalur optimal yang melewati tepian jurang jalur terpendek. Ini karena Q-Learning selalu mengevaluasi berdasarkan *action* terbaik (*greedy*) dan tidak memperhitungkan risiko bahwa *agent* mungkin secara tidak sengaja jatuh ke jurang akibat eksplorasi ϵ -greedy.
- **SARSA** menemukan jalur yang lebih aman jalur yang lebih panjang tetapi menjauh dari tepian jurang. Ini karena SARSA memperhitungkan bahwa dengan strategi ϵ -greedy, *agent* mungkin memilih *action* acak yang membawanya jatuh ke jurang saat berada di tepian. SARSA menilai *state* di tepian jurang secara lebih rendah karena memperhitungkan risiko eksplorasi ini.

Implikasi praktis Q-Learning lebih cocok ketika *policy* eksplorasi dan *policy* yang dipelajari dapat berbeda (misalnya, saat eksplorasi di simulasi aman). SARSA lebih

cocok ketika keamanan selama proses pembelajaran itu penting (misalnya, robot fisik yang tidak boleh menabrak saat belajar, atau sistem keuangan di mana kerugian besar harus dihindari) [1].

3.4 Deep Q-Network (DQN)

Deep Q-Network (DQN) merupakan pengembangan revolusioner dari Q-Learning yang diperkenalkan oleh Volodymyr Mnih dan rekan-rekannya di DeepMind. Versi awal dipublikasikan dalam *workshop paper* pada tahun 2013, dan versi final yang lebih lengkap diterbitkan di jurnal *Nature* pada Februari 2015 [10]. Publikasi ini dianggap sebagai salah satu momen paling berpengaruh dalam sejarah modern AI, karena mendemonstrasikan untuk pertama kalinya bahwa satu sistem RL tunggal mampu belajar menguasai berbagai tugas kompleks secara langsung dari data sensorik mentah (piksel layar). DQN menggabungkan kekuatan representasi *deep neural network* dengan fondasi algoritma Q-Learning, melahirkan bidang yang kini dikenal sebagai Deep Reinforcement Learning (Deep RL).

3.4.1 Motivasi dan Prinsip Kerja

Keterbatasan fundamental dari Q-Learning tradisional (dan SARSA) terletak pada representasi *Q-table*-nya. Dalam *Q-table*, setiap pasangan *state-action* memerlukan satu entri terpisah di memori. Ini hanya praktis untuk masalah dengan *state space* dan *action space* yang kecil dan diskret. Namun, banyak masalah dunia nyata memiliki *state space* yang sangat besar atau bahkan kontinu:

- Pada permainan video dengan layar berukuran 84×84 piksel dalam *grayscale* (256 level intensitas per piksel), jumlah kemungkinan *state* adalah $256^{84 \times 84}$ angka yang jauh melampaui jumlah atom di alam semesta.
- Pada masalah robotik, *state* berupa sudut dan kecepatan sendi menghasilkan *state space* kontinu yang tak terhingga.

Menyimpan tabel untuk seluruh *state* ini jelas tidak mungkin. DQN mengatasi masalah ini dengan mengganti *Q-table* menggunakan *deep neural network* sebagai function approximator (pengaproksimasi fungsi) [10]. Alih-alih menyimpan nilai Q untuk setiap pasangan *state-action* secara terpisah, *neural network* belajar menggeneralisasi ia mempelajari pola umum yang memungkinkannya mengestimasi $Q(s, a; \theta)$ bahkan untuk *state* yang belum pernah dilihat sebelumnya. Parameter θ merepresentasikan bobot (*weights*) dari *neural network*.

Namun, menggabungkan neural network dengan Q-Learning bukanlah hal yang mudah. Sebelum DQN, upaya-upaya sebelumnya untuk menggunakan neural network sebagai *function approximator* dalam RL sering gagal karena pelatihan menjadi tidak stabil dan tidak konvergen. Ketidakstabilan ini disebabkan oleh dua masalah utama: (1) korelasi temporal yang kuat antar data pelatihan berurutan, dan (2) perubahan target Q yang terus bergerak seiring pembaruan bobot. DQN mengatasi kedua masalah ini dengan dua inovasi kunci yang dibahas di sub bab berikutnya.

3.4.2 Inovasi Kunci DQN

DQN memperkenalkan dua teknik penting yang menjadi kunci keberhasilannya dalam menstabilkan pelatihan neural network untuk RL [10]:

a. Experience Replay

Masalah pertama yang dihadapi saat melatih neural network dengan data RL adalah korelasi temporal data pengalaman yang dikumpulkan secara berurutan sangat berkorelasi satu sama lain. Misalnya, jika *agent* bermain di level tertentu suatu permainan selama 100 langkah, maka 100 sampel pelatihan berurutan tersebut sangat mirip satu sama lain (karena *state* berubah secara perlahan). Melatih neural network pada data yang sangat berkorelasi ini menyebabkan *overfitting* lokal dan ketidakstabilan. *Experience replay* mengatasi masalah ini dengan pendekatan yang elegan: *agent* menyimpan setiap pengalamannya berupa transisi $(S_t, A_t, R_{t+1}, S_{t+1})$ ke dalam sebuah *replay buffer* (memori siklik dengan kapasitas terbatas, biasanya ratusan ribu hingga jutaan transisi). Saat proses pembelajaran, alih-alih belajar dari pengalaman terbaru secara langsung, *agent* mengambil mini-batch transisi secara acak dari *buffer* ini dan menggunakannya untuk memperbarui bobot neural network.

Teknik ini memiliki beberapa keuntungan penting:

- **Memutus korelasi temporal:** Karena sampel diambil secara acak dari berbagai titik waktu, data pelatihan tidak lagi berkorelasi kuat, mirip dengan asumsi *i.i.d.* (independent and identically distributed) yang dibutuhkan oleh algoritma *machine learning* standar.
- **Efisiensi data:** Setiap pengalaman dapat digunakan berkali-kali untuk pelatihan, bukan hanya sekali saat terjadi. Ini sangat meningkatkan efisiensi

penggunaan data, yang penting mengingat interaksi dengan *environment* bisa mahal (secara komputasi maupun sumber daya fisik).

- **Menghindari catastrophic forgetting:** Tanpa *replay*, *agent* yang belajar dari pengalaman terbaru saja bisa "melupakan" apa yang dipelajari dari pengalaman lama.

b. Target Network

Masalah kedua adalah target yang bergerak (*moving target*). Dalam Q-Learning, target yang digunakan untuk melatih neural network adalah $R_{\{t+1\}} + \gamma \max_a Q(S_{\{t+1\}}, a; \theta)$. Masalahnya, target ini sendiri bergantung pada parameter θ yang sedang diperbarui seperti anjing yang mengejar ekornya sendiri. Setiap kali kita memperbarui θ untuk meningkatkan $Q(S_t, A_t)$, target $Q(S_{\{t+1\}}, a; \theta)$ juga ikut berubah, yang menyebabkan osilasi dan ketidakstabilan dalam pelatihan.

DQN mengatasi masalah ini dengan menggunakan dua jaringan neural terpisah yang memiliki arsitektur identik tetapi bobot yang berbeda:

- **Main Network** (Q-network, parameter θ): Jaringan utama yang digunakan untuk memilih *action* (menentukan *policy*) dan yang bobotnya diperbarui secara terus-menerus di setiap langkah pelatihan melalui *gradient descent*.
- **Target Network** (parameter θ^-): Salinan "beku" dari *main network* yang hanya digunakan untuk menghitung nilai target Q dalam persamaan pembaruan. Bobot target network tidak diperbarui secara terus-menerus, melainkan disinkronkan (*copied*) dari main network secara periodik (misalnya, setiap 10.000 langkah). Ini berarti target Q tetap stabil selama ribuan langkah pelatihan, memberikan " pijakan " yang stabil untuk proses optimasi.

Dengan target network, target pembaruan menjadi: $Y = R_{\{t+1\}} + \gamma \max_a Q(S_{\{t+1\}}, a; \theta^-)$, di mana θ^- adalah bobot target network yang tetap (*frozen*) selama periode tertentu. Ini secara signifikan mengurangi osilasi dan meningkatkan stabilitas pelatihan, karena target tidak lagi berubah-ubah setiap kali main network diperbarui [10].

3.4.3 Langkah-Langkah Algoritma DQN

Berikut adalah prosedur lengkap algoritma DQN [10]:

Algoritma: Deep Q-Network (DQN)

Input: Kapasitas replay buffer N , ukuran mini-batch m ,
frekuensi pembaruan target network C ,
learning rate α , discount factor γ , exploration rate ϵ

1. INISIALISASI:

- a. Inisialisasi main Q-network dengan bobot acak θ
- b. Inisialisasi target Q-network: $\theta^- \leftarrow \theta$
- c. Inisialisasi replay buffer D dengan kapasitas N

2. UNTUK setiap episode:

- a. Dapatkan state awal S_1 (preprocessing jika diperlukan)

b. UNTUK setiap langkah t dalam episode:

- i. PILIH action A_t menggunakan ϵ -greedy:
 - Dengan probabilitas ϵ : pilih action acak
 - Dengan probabilitas $(1-\epsilon)$: pilih $\operatorname{argmax}_a Q(S_t, a; \theta)$
- ii. EKSEKUSI action A_t
Amati reward R_{t+1} dan state baru S_{t+1}
- iii. SIMPAN transisi $(S_t, A_t, R_{t+1}, S_{t+1})$ ke replay buffer D
- iv. SAMPLING: Ambil mini-batch m transisi secara acak dari D
- v. HITUNG target Q-value untuk setiap transisi (S_j, A_j, R_j, S'_j) :
Jika S'_j terminal: $Y_j = R_j$
Jika S'_j non-terminal: $Y_j = R_j + \gamma \max_a Q(S'_j, a; \theta^-)$
- vi. PERBARUI main network θ :
Minimisasi loss $L(\theta) = (1/m) \sum (Y_j - Q(S_j, A_j; \theta))^2$

menggunakan gradient descent

vii. Setiap C langkah: PERBARUI target network $\theta^- \leftarrow \theta$

3. ULANGI langkah 2 sampai konvergen

3.5 Perbandingan Algoritma

Berikut adalah perbandingan ketiga algoritma yang telah dibahas [1][8][10]:

Aspek	Q-Learning	SARSA	DQN
Tipe Policy	<i>Off-policy</i>	<i>On-policy</i>	<i>Off-policy</i>
Representasi Nilai	Q-Table	Q-Table	Neural Network
Target Update	$\max_a Q(S', a)$	$Q(S', A')$	$\max_a Q(S', a; \theta^-)$
State Space	Diskret, kecil	Diskret, kecil	Besar, kontinu
Kompleksitas	Rendah	Rendah	Tinggi
Stabilitas Training	Sedang	Tinggi	Sedang (dengan <i>experience replay</i> & <i>target network</i>)
Perilaku	Agresif (cari optimal)	Konservatif (cari aman)	Agresif (cari optimal)
Kebutuhan Memori	Rendah	Rendah	Tinggi (<i>replay buffer</i> , neural network)
Contoh Penerapan	<i>Grid world</i> , navigasi sederhana	Navigasi dengan risiko	Permainan video, kontrol kompleks

BAB IV

STUDI KASUS

4.1 Studi Kasus 1: ChatGPT dan Reinforcement Learning from Human Feedback

4.1.1 Latar Belakang

Salah satu penerapan *reinforcement learning* yang paling berpengaruh dan paling dikenal luas dalam beberapa tahun terakhir adalah penggunaannya dalam melatih model bahasa besar (*Large Language Model/LLM*) seperti ChatGPT dari OpenAI. Pada tahun 2022, OpenAI mempublikasikan makalah berjudul "*Training Language Models to Follow Instructions with Human Feedback*" yang memperkenalkan InstructGPT pendahulu langsung dari ChatGPT [2]. Makalah ini mendemonstrasikan teknik yang disebut *Reinforcement Learning from Human Feedback* (RLHF), di mana RL digunakan untuk menyelaraskan (*align*) perilaku model bahasa agar lebih bermanfaat (*helpful*), jujur (*honest*), dan aman (*harmless*) sesuai dengan preferensi manusia.

Sebelum RLHF, model bahasa besar dilatih dengan metode *supervised learning* standar memprediksi kata berikutnya dari teks yang ada. Meskipun menghasilkan teks yang fasih secara tata bahasa, model ini sering menghasilkan output yang tidak relevan, berbelit-belit, mengandung informasi salah, atau bahkan berbahaya. RLHF menjadi terobosan kunci karena memberikan mekanisme untuk "mengarahkan" model agar menghasilkan respons yang benar-benar diinginkan pengguna, bukan sekadar respons yang secara statistik paling mungkin [2].

ChatGPT, yang diluncurkan pada November 2022, menjadi fenomena global dengan lebih dari 100 juta pengguna aktif dalam dua bulan pertama menjadikannya aplikasi yang paling cepat tumbuh dalam sejarah. Keberhasilan ini sebagian besar berkat teknik RLHF yang membuat interaksi dengan model bahasa terasa jauh lebih alami dan bermanfaat.

4.1.2 Formulasi Masalah sebagai RL

Proses pelatihan RLHF pada InstructGPT/ChatGPT diformulasikan ke dalam kerangka RL sebagai berikut [2]:

- **Agent:** Model bahasa (LLM) yang menghasilkan teks sebagai respons terhadap instruksi pengguna. *Agent* ini memiliki miliaran parameter yang menentukan bagaimana ia memproses dan menghasilkan teks.
- **Environment:** Konteks percakapan yang diberikan oleh pengguna, termasuk *prompt* (instruksi), riwayat percakapan, dan preferensi manusia. Lingkungan ini bersifat unik karena berbeda dari lingkungan RL tradisional interaction terjadi dalam ruang bahasa alami.
- **State:** *Prompt* atau instruksi yang diberikan pengguna beserta konteks percakapan saat ini. Setiap sekuen token yang telah di-*generate* sejauh ini juga menjadi bagian dari *state*.
- **Action:** Token (kata atau sub-kata) yang dipilih oleh model pada setiap langkah generasi teks. *Action space*-nya sangat besar puluhan ribu token dalam kosakata model.
- **Reward:** Skor yang diberikan oleh Reward Model sebuah model terpisah yang telah dilatih untuk memprediksi preferensi manusia. Skor yang lebih tinggi menunjukkan respons yang lebih disukai oleh manusia.

4.1.3 Tahapan Pelatihan RLHF

Proses RLHF pada InstructGPT terdiri dari tiga tahap yang saling terkait [2]:

Tahap 1: Supervised Fine-Tuning (SFT): Tahap pertama mirip dengan *supervised learning* konvensional. Tim manusia ahli (*human labelers*) membuat dataset yang berisi pasangan *prompt*-respons berkualitas tinggi. Model bahasa dasar (*base model*, misalnya GPT-3) kemudian di-*fine-tune* pada dataset ini agar belajar format dan gaya respons yang diinginkan. Hasilnya adalah model SFT yang mampu mengikuti instruksi dasar, meskipun belum sempurna.

Tahap 2: Pelatihan Reward Model (RM): Tahap kedua adalah langkah krusial yang membedakan RLHF dari metode konvensional. Karena manusia tidak mungkin secara manual mengevaluasi setiap respons yang dihasilkan selama pelatihan RL (jumlahnya jutaan), maka dilatih sebuah model terpisah Reward Model yang bertindak sebagai "pengganti" penilaian manusia:

1. Model SFT menghasilkan beberapa respons berbeda untuk *prompt* yang sama.
2. Anotator manusia meranking respons-respons tersebut dari yang terbaik hingga terburuk berdasarkan kualitas, keakuratan, dan keamanannya.

3. Data ranking ini digunakan untuk melatih Reward Model, yang belajar memprediksi skor numerik (*scalar reward*) untuk setiap pasangan *prompt*-respons. Skor yang lebih tinggi berarti respons yang lebih disukai manusia.

Reward Model ini pada dasarnya adalah aproksimasi dari "fungsi reward" dalam terminologi RL ia mengkuantifikasi preferensi manusia ke dalam sinyal numerik yang dapat dioptimalkan oleh algoritma RL.

Tahap 3: Optimasi dengan Reinforcement Learning (PPO): Tahap terakhir adalah tahap RL yang sesungguhnya, di mana model bahasa dioptimalkan menggunakan algoritma Proximal Policy Optimization (PPO) sebuah algoritma *policy gradient* yang modern dan stabil [2]:

1. Model bahasa (yang kini berperan sebagai "policy" π) menerima *prompt* dan menghasilkan respons.
2. Reward Model mengevaluasi respons tersebut dan memberikan skor reward.
3. Algoritma PPO menggunakan skor reward ini untuk memperbarui parameter model bahasa meningkatkan probabilitas menghasilkan respons yang serupa dengan yang mendapat skor tinggi, dan menurunkan probabilitas respons berskor rendah.
4. Untuk mencegah model menyimpang terlalu jauh dari kemampuan bahasa aslinya (yang bisa menyebabkan output tidak koheren), diterapkan penalti KL-divergence sebuah batasan yang memastikan *policy* yang diperbarui tidak terlalu berbeda dari model SFT awal.

Proses ini diulangi jutaan kali dengan berbagai *prompt* yang berbeda, secara bertahap membentuk perilaku model agar semakin selaras dengan preferensi manusia.

4.1.4 Hasil

Hasil evaluasi InstructGPT menunjukkan peningkatan yang sangat signifikan [2]:

- Model InstructGPT dengan 1.3 miliar parameter ($13\times$ lebih kecil) dinilai lebih baik oleh anotator manusia dibandingkan model GPT-3 standar dengan 175 miliar parameter. Ini menunjukkan bahwa RLHF sangat efisien model kecil yang diselaraskan bisa mengalahkan model besar yang tidak diselaraskan.
- Anotator manusia lebih memilih output InstructGPT dibandingkan GPT-3 pada 85% kasus yang diuji.

- InstructGPT menunjukkan penurunan signifikan dalam menghasilkan informasi yang salah (*hallucination*), konten berbahaya (*toxic content*), dan respons yang tidak relevan.
- Model juga menunjukkan peningkatan kemampuan mengikuti instruksi yang kompleks dan multi-langkah.

4.1.5 Keterkaitan dengan Teori

Studi kasus RLHF pada ChatGPT/InstructGPT mendemonstrasikan penerapan konsep-konsep teori RL secara unik dan inovatif:

- **MDP:** Proses generasi teks diformulasikan sebagai pengambilan keputusan sekuensial setiap pemilihan token adalah sebuah *action* yang mengubah *state* (teks yang sudah dihasilkan) dan pada akhirnya menentukan *reward* (penilaian kualitas respons).
- **Policy (π):** Model bahasa itu sendiri berfungsi sebagai *policy* ia memetakan *state* (prompt + konteks) ke distribusi probabilitas atas *action* (token berikutnya). RLHF mengoptimalkan *policy* ini agar selaras dengan preferensi manusia.
- **Reward Function:** Reward Model bertindak sebagai aproksimasi fungsi reward yang merepresentasikan preferensi manusia. Ini merupakan solusi kreatif terhadap tantangan mendefinisikan *reward function* untuk tugas yang subjektif seperti "kualitas bahasa".
- **Eksplorasi vs. Eksploitasi:** Penalti KL-divergence menjaga keseimbangan antara eksploitasi (menghasilkan respons berskor tinggi menurut Reward Model) dan eksplorasi (mempertahankan keragaman dan koherensi bahasa dari model asli).

4.2 Studi Kasus 2: Reinforcement Learning pada Sistem Rekomendasi (YouTube)

4.2.1 Latar Belakang

Sistem rekomendasi (*recommendation system*) merupakan salah satu teknologi yang paling berpengaruh dalam kehidupan digital sehari-hari. Platform seperti YouTube, Netflix, TikTok, dan Spotify menggunakan sistem rekomendasi untuk memutuskan konten apa yang ditampilkan kepada miliaran pengguna mereka setiap hari. Dalam konteks ini, YouTube

menjadi salah satu pelopor penerapan *reinforcement learning* dalam sistem rekomendasi skala besar produksi.

Pada tahun 2019, Chen *et al.* dari Google Brain/YouTube mempublikasikan makalah berjudul "*Top-K Off-Policy Correction for a REINFORCE Recommender System*" yang menguraikan bagaimana YouTube menerapkan algoritma RL (khususnya varian REINFORCE, sebuah algoritma *policy gradient*) untuk sistem rekomendasi video produksi yang melayani lebih dari dua miliar pengguna [3]. Pendekatan ini menandai pergeseran paradigma penting: dari sistem rekomendasi berbasis *supervised learning* yang mengoptimalkan metrik jangka pendek (seperti *click-through rate*/CTR), menuju sistem berbasis RL yang mengoptimalkan kepuasan pengguna jangka panjang (*long-term user satisfaction*).

Sebelumnya, pada tahun 2018, Zhao *et al.* juga telah mengeksplorasi penggunaan DQN dan *Actor-Critic* untuk sistem rekomendasi dalam makalah "*Deep Reinforcement Learning for List-wise Recommendations*" [9], yang memberikan kerangka kerja dasar untuk merumuskan rekomendasi sebagai masalah RL.

4.2.2 Formulasi Masalah sebagai RL

Sistem rekomendasi berbasis RL memformulasikan interaksi pengguna-platform ke dalam kerangka MDP sebagai berikut [3][9]:

- **Agent:** Algoritma sistem rekomendasi (model RL) yang menentukan konten apa yang ditampilkan kepada pengguna.
- **Environment:** Pengguna yang berinteraksi dengan platform. Setiap pengguna memiliki preferensi yang unik, dinamis (berubah seiring waktu), dan tidak sepenuhnya dapat diamati membuat lingkungan ini bersifat *partially observable* dan sangat kompleks.
- **State:** Representasi dari konteks interaksi saat ini, yang mencakup: riwayat tontonan pengguna (*watch history*), profil pengguna (usia, lokasi, perangkat), waktu hari, video yang baru saja ditonton, interaksi terkini (like, dislike, skip), dan fitur kontekstual lainnya. State ini di-*encode* menjadi representasi vektor berdimensi tinggi oleh neural network.
- **Action:** Pemilihan satu atau beberapa video (*top-K items*) dari katalog yang berisi jutaan video untuk direkomendasikan kepada pengguna. Ukuran *action*

space yang sangat besar (jutaan item) menjadi tantangan teknis utama yang membedakan masalah ini dari aplikasi RL tradisional.

- **Reward:** Sinyal umpan balik dari pengguna yang mencerminkan kepuasannya.

Reward dapat berupa:

1. *Immediate reward:* klik pada video, like, share, subscribe.
2. *Long-term reward:* total waktu tonton (*watch time*), frekuensi kembali ke platform, retensi pengguna jangka panjang.

4.2.3 Mengapa RL Lebih Baik dari Supervised Learning untuk Rekomendasi?

Sebelum RL, sebagian besar sistem rekomendasi menggunakan *supervised learning* misalnya, melatih model untuk memprediksi apakah pengguna akan mengklik suatu item (*click prediction*). Namun, pendekatan ini memiliki beberapa kelemahan fundamental [3][9]:

1. **Optimasi jangka pendek (*myopic*):** *Supervised learning* mengoptimalkan metrik sesaat seperti klik, tanpa mempertimbangkan dampak jangka panjang. Misalnya, konten sensasional (*clickbait*) mungkin mendapat banyak klik tetapi membuat pengguna kecewa setelah menontonnya dan meninggalkan platform.
2. **Tidak memodelkan dampak sekuensial:** Rekomendasi saat ini mempengaruhi preferensi pengguna di masa depan. Jika pengguna terus-menerus ditunjukkan konten sejenis, preferensinya bisa menyempit (*filter bubble*). *Supervised learning* tidak mempertimbangkan efek ini.
3. **Feedback loop:** Sistem rekomendasi hanya mendapat umpan balik tentang item yang *direkomendasikan* ia tidak tahu bagaimana pengguna akan bereaksi terhadap item yang *tidak ditampilkan*. RL secara alami menangani masalah eksplorasi ini.

RL mengatasi kelemahan-kelemahan ini karena memformulasikan rekomendasi sebagai *pengambilan keputusan sekuensial* yang mempertimbangkan dampak jangka panjang melalui *discount factor* dan akumulasi *reward* [3]

4.2.4 Arsitektur dan Metode

Sistem rekomendasi berbasis RL YouTube menggunakan beberapa teknik kunci [3]:

- a. **REINFORCE dengan Top-K Off-Policy Correction**

YouTube menggunakan algoritma REINFORCE (sebuah algoritma *policy gradient*, varian dari RL) yang dimodifikasi untuk menangani tantangan-tantangan spesifik sistem rekomendasi skala besar:

- **Masalah *action space* besar:** Dengan jutaan video di katalog, standard RL tidak bisa langsung diterapkan. YouTube menggunakan teknik *softmax* atas *embedding* item untuk menangani *action space* yang sangat besar ini.
- **Off-policy correction:** Karena data pelatihan dikumpulkan dari versi model sebelumnya (bukan model yang sedang dilatih), diperlukan koreksi *importance sampling* untuk mengatasi bias. Paper ini memperkenalkan teknik *top-K off-policy correction* yang secara spesifik menangani kasus di mana sistem merekomendasikan beberapa item sekaligus (bukan hanya satu).
- **Long-term reward optimization:** Alih-alih mengoptimalkan klik sesaat, model dilatih untuk memaksimalkan metrik kepuasan jangka panjang seperti total waktu tonton dan retensi pengguna.

b. Arsitektur Neural Network

Model rekomendasi menggunakan neural network yang mengambil fitur-fitur kontekstual (riwayat pengguna, fitur video, konteks temporal) sebagai input dan menghasilkan distribusi probabilitas atas video yang harus direkomendasikan. Arsitektur ini memungkinkan generalisasi ke item baru dan pengguna baru yang belum pernah dilihat sebelumnya.

c. Eksplorasi dalam Sistem Rekomendasi

Eksplorasi merupakan aspek yang sangat krusial dalam sistem rekomendasi. Tanpa eksplorasi, sistem akan selalu merekomendasikan jenis konten yang sama, menciptakan *filter bubble* dan mengurangi keragaman. Namun, eksplorasi yang berlebihan menampilkan konten tidak relevan yang membuat pengguna pergi. YouTube menyeimbangkan ini melalui *stochastic policy* yang memberikan probabilitas kecil untuk merekomendasikan konten di luar preferensi yang sudah diketahui [3].

4.2.5 Hasil

Penerapan RL pada sistem rekomendasi YouTube memberikan hasil yang signifikan [3]:

- Peningkatan bermakna (*statistically significant improvement*) pada metrik kepuasan pengguna jangka panjang dibandingkan dengan *baseline* berbasis *supervised learning*.
- Peningkatan retensi pengguna pengguna yang menerima rekomendasi dari model RL lebih cenderung kembali ke platform di hari-hari berikutnya.
- Peningkatan keragaman konten yang direkomendasikan, mengurangi efek *filter bubble* yang sering menjadi kritik terhadap sistem rekomendasi tradisional.
- Sistem berhasil dioperasikan dalam skala produksi yang melayani lebih dari dua miliar pengguna, membuktikan bahwa RL dapat diterapkan pada masalah dunia nyata yang sangat besar.

4.2.6 Keterkaitan dengan Teori

Studi kasus sistem rekomendasi mendemonstrasikan penerapan konsep RL pada masalah bisnis dunia nyata yang kompleks:

- **MDP:** Interaksi pengguna-platform diformulasikan sebagai MDP, di mana *state* berubah seiring pengguna berinteraksi dengan konten yang direkomendasikan, dan setiap rekomendasi (*action*) mempengaruhi *state* dan *reward* selanjutnya.
- **Reward Function:** Desain *reward* menjadi tantangan kunci apakah metrik *engagement* (waktu tonton) atau metrik *satisfaction* (survei kepuasan) yang harus dioptimalkan? Ini mencerminkan tantangan *reward shaping* yang telah dibahas di Sub Bab 2.2.5.
- **Eksplorasi vs. Eksploitasi:** Dilema ini sangat nyata merekomendasikan konten yang sudah diketahui disukai (eksploitasi) vs. memperkenalkan konten baru yang mungkin memperluas minat pengguna (eksplorasi). Sistem rekomendasi YouTube menggunakan *stochastic policy* untuk menyeimbangkan keduanya.
- **Off-Policy Learning:** YouTube menggunakan data dari sistem rekomendasi versi sebelumnya (*behavior policy*) untuk melatih model baru (*target policy*) ini merupakan penerapan langsung dari konsep *off-policy learning* yang telah dibahas di Sub Bab 3.1.
- **Discount Factor:** Penggunaan *discount factor* memungkinkan sistem menghargai kepuasan jangka pendek (klik sekarang) dan jangka panjang (retensi minggu depan) secara bersamaan.

4.3 Keterkaitan Studi Kasus dengan Teori

Kedua studi kasus di atas mendemonstrasikan penerapan teori RL di luar domain permainan, pada masalah dunia nyata yang langsung mempengaruhi kehidupan miliaran pengguna:

Konsep Teori	ChatGPT/RLHF	Sistem Rekomendasi YouTube
Agent	Model bahasa (LLM)	Algoritma rekomendasi
Environment	Konteks percakapan + manusia	Pengguna platform
State	Prompt + konteks percakapan	Riwayat + profil pengguna
Action	Pemilihan token (kata)	Pemilihan video (top-K)
Reward	Skor dari Reward Model	Engagement pengguna
MDP	✓ (Generasi teks sekuensial)	✓ (Interaksi sekuensial)
Policy	LLM sebagai $\pi(a s)$	Distribusi probabilitas item
Eksplorasi	KL-penalty constraint	Stochastic policy
Algoritma	PPO (Policy Gradient)	REINFORCE + Off-policy correction
Tantangan Unik	Reward subjektif (preferensi manusia)	Action space sangat besar (jutaan item)

Kedua studi kasus ini menunjukkan bahwa RL bukan hanya teknik untuk bermain *game*, tetapi kerangka kerja yang sangat powerful untuk memecahkan masalah pengambilan keputusan sekuensial di berbagai domain dari penyelarasan model bahasa yang digunakan ratusan juta orang, hingga personalisasi konten untuk miliaran pengguna platform digital.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan pembahasan yang telah diuraikan dalam makalah ini, dapat ditarik beberapa kesimpulan sebagai berikut:

1. **Teori Dasar RL:** *Reinforcement learning* adalah paradigma *machine learning* yang mempelajari bagaimana sebuah *agent* mengambil keputusan optimal dalam suatu *environment* melalui proses *trial-and-error*. Landasan teoritisnya bertumpu pada *Markov Decision Process* (MDP), yang terdiri dari *state space*, *action space*, *transition function*, *reward function*, dan *discount factor*. Konsep *policy* dan *value function* (baik *state-value* $V(s)$ maupun *action-value* $Q(s,a)$) memungkinkan *agent* untuk mengevaluasi dan menyempurnakan strateginya secara iteratif melalui Persamaan Bellman. Dilema eksplorasi vs. eksploitasi juga merupakan tantangan fundamental yang harus diatasi dalam setiap penerapan RL.
2. **Langkah-Langkah dan Algoritma:** Tiga algoritma utama yang dibahas Q-Learning, SARSA, dan DQN menunjukkan evolusi dalam pendekatan RL. Q-Learning sebagai algoritma *off-policy* mempelajari *policy* optimal secara langsung menggunakan operator *max* dalam pembaruan Q-value. SARSA sebagai algoritma *on-policy* menghasilkan kebijakan yang lebih konservatif dan aman karena memperhitungkan risiko eksplorasi. DQN merevolusi RL dengan menggabungkan Q-Learning dan *deep neural network*, ditambah inovasi *experience replay* dan *target network*, sehingga mampu menangani masalah dengan *state space* yang sangat besar yang tidak mungkin ditangani oleh algoritma tabular.
3. **Studi Kasus:** Studi kasus RLHF pada ChatGPT/InstructGPT menunjukkan bahwa RL dapat digunakan untuk menyelaraskan model bahasa besar dengan preferensi manusia, menghasilkan model yang $13\times$ lebih kecil namun dinilai lebih baik oleh manusia. Sementara itu, studi kasus sistem rekomendasi YouTube mendemonstrasikan bahwa RL dapat dioperasikan dalam skala produksi yang melayani lebih dari dua miliar pengguna, dengan fokus pada optimasi kepuasan jangka panjang alih-alih metrik sesaat. Kedua studi kasus ini membuktikan bahwa RL bukan hanya teknik untuk domain permainan, melainkan kerangka kerja yang sangat luas penerapannya dalam kehidupan nyata.

5.2 Saran

Berikut adalah beberapa saran untuk pengembangan lebih lanjut:

1. Eksplorasi algoritma RL yang lebih modern seperti *Proximal Policy Optimization* (PPO), *Soft Actor-Critic* (SAC), dan *Direct Preference Optimization* (DPO) penerus RLHF yang lebih efisien dapat memperluas cakupan pemahaman mengenai perkembangan terkini di bidang RL.
2. Penerapan RL pada domain lain seperti robotika, kendaraan otonom, pengelolaan energi di pusat data, dan penemuan obat baru merupakan area penelitian yang sangat menjanjikan.
3. Eksplorasi tantangan dan keterbatasan RL, seperti masalah *sample efficiency*, *reward hacking*, *sim-to-real transfer*, dan keamanan (*safety*) dalam penerapan RL pada sistem dunia nyata, penting untuk memahami batasan dan risiko teknologi ini.
4. Implementasi praktis algoritma RL menggunakan *library* seperti OpenAI Gym, Stable Baselines3, atau RLlib, serta eksplorasi teknik RLHF menggunakan framework *TRL* (*Transformer Reinforcement Learning*), dapat memberikan pengalaman langsung dalam menerapkan konsep-konsep yang telah dipelajari.

DAFTAR PUSTAKA

- [1] R. S. Sutton dan A. G. Barto, *Reinforcement Learning: An Introduction*, edisi ke-2. Cambridge, MA, USA: MIT Press, 2018.
- [2] L. Ouyang *et al.*, "Training language models to follow instructions with human feedback," dalam *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, hal. 27730–27744, 2022.
- [3] M. Chen *et al.*, "Top-K off-policy correction for a REINFORCE recommender system," dalam *Proceedings of the 12th ACM International Conference on Web Search and Data Mining (WSDM)*, hal. 456–464, 2019, doi: 10.1145/3289600.3290999.
- [4] B. F. Skinner, *The Behavior of Organisms: An Experimental Analysis*. New York, NY, USA: Appleton-Century-Crofts, 1938.
- [5] L. P. Kaelbling, M. L. Littman, dan A. W. Moore, "Reinforcement learning: A survey," *Journal of Artificial Intelligence Research*, vol. 4, hal. 237–285, 1996, doi: 10.1613/jair.301.
- [6] R. Bellman, *Dynamic Programming*. Princeton, NJ, USA: Princeton University Press, 1957.
- [7] K. Arulkumaran, M. P. Deisenroth, M. Brundage, dan A. A. Bharath, "Deep reinforcement learning: A brief survey," *IEEE Signal Processing Magazine*, vol. 34, no. 6, hal. 26–38, Nov. 2017, doi: 10.1109/MSP.2017.2743240.
- [8] C. J. C. H. Watkins dan P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3–4, hal. 279–292, Mei 1992, doi: 10.1007/BF00992698.
- [9] X. Zhao, L. Zhang, Z. Ding, L. Xia, J. Tang, dan D. Yin, "Recommendations with negative feedback via pairwise deep reinforcement learning," dalam *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, hal. 1040–1048, 2018, doi: 10.1145/3219819.3219886.
- [10] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, hal. 529–533, Feb. 2015, doi: 10.1038/nature14236.